# MAKING REAL LIFE JARVIS

BY PULSE OF AI

⚠️ ADVICE: Please ensure you watch all the videos and extract code from the pinned comments or Jarvis might not work.

## VIDEO MASTER ARCHIVE

PART 1 // WATCH

PART 2 // WATCH

PART 3 // WATCH

PART 4 // WATCH

PART 5 // WATCH

PART 6 // WATCH

PART 7 // WATCH

PART 8 // WATCH

PART 9 // WATCH

PART 10 // WATCH

PART 11 // WATCH

PART 12 // WATCH

PART 13 // WATCH

PART 14 // WATCH

PART 15 // WATCH

PART 16 // WATCH

PART 17 // WATCH

PART 18 // WATCH

PART 19 // WATCH

PART 20 // WATCH

PART 21 // WATCH

PART 22 // WATCH

PART 23 // WATCH

## SOURCE CODE DOSSIER

FILE: agent.py

```
"""
agent.py
Real-Time Gemini Live Engine with Semi-Transparent Cyberpunk HUD Console (Episode
23)
Autonomous App Orchestration, Multi-Tab Web Swarm, PPT/PDF Engine & Master Serious
Mode Pipeline
Built by Swayam Dhawale, Founder of Pulse of AI.
"""

import asyncio
import datetime
import os
import sys
import time
import math
import struct
import threading
import subprocess
import collections
import tkinter as tk
from tkinter import scrolledtext
import pyaudio
import winsound
import keyboard
import requests
from dotenv import load_dotenv
from google import genai
from google.genai import errors, types
import websockets
from rich.console import Console
import fui_bridge

load_dotenv()
if sys.platform == "win32":
    try:
        sys.stdout.reconfigure(encoding="utf-8")
        sys.stderr.reconfigure(encoding="utf-8")
    except Exception:
        pass
console = Console()

# --- REMOTE KILL-SWITCH (Pulse of AI) ---
STATUS_URL =
"https://gist.githubusercontent.com/PulseOfAIDev/a9951c6e858aacbabb604a0ed23f6ab7/raw/

def verify_app_access():
    """Checks remote status from GitHub Gist and displays cloud-configured
message."""
    try:
        fresh_url = f"{STATUS_URL}?t={int(time.time())}"
        response = requests.get(fresh_url, headers={"Cache-Control": "no-cache"},
timeout=4).json()

        status = str(response.get("app_status", "free")).strip().lower()
        message = response.get(
            "message",
            "The free beta access period has concluded. To activate a subscription
tier or upgrade your account, visit: https://pulseofaidev.github.io"
        )

        if status != "free":
            console.print(f"\n[bold red]🛑 [ACCESS RESTRICTED][/bold red]
{message}\n")
            sys.exit(0)
    except Exception:
```

```
        pass

# Run verification immediately on boot
verify_app_access()

# --- SECURE KEYRING INTEGRATION ---
from Setup import get_or_prompt_api_key

# Hardware & Telemetry
from hardware_control import (
    get_brightness,
    get_volume,
    set_brightness,
    set_volume,
    get_battery_status,
    get_system_vitals,
    get_system_temperatures,
    restore_master_volume,
)

# Software, Media Telemetry & Navigation Tools
from software_control import (
    open_website,
    open_application,
    close_application,
    purge_background_tasks,
    control_media,
    get_current_media_info,
    is_media_actively_playing,
    navigate_shorts,
    skip_video,
    seek_video,
    toggle_video_view,
)

# Local Zero-Latency Vision & Autonomous Grounding Tools
from screen_vision import (
    analyze_screen,
    find_ui_element,
    skip_youtube_ad,
)

# Self-Modifying Tri-Core Dynamic Skill Synthesizer
from skill_synthesizer import (
    evolve_skill,
    list_active_skills,
)

# Fast Cloud Reasoning Tool
from llm_router import (
    query_deepseek_reasoning,
)

# Episode 17: Autonomous App Composer & Typing Engine
from app_automator import (
    open_and_compose,
)

# Episode 18 & 19: Autonomous PowerPoint Generator & Presentation Engine
from ppt_engine import (
    create_and_launch_ppt,
)

# Episode 19 & 20: Autonomous Executive PDF Briefing Engine & Episode 23 Master
Pipeline
from pdf_engine import (
    research_and_generate_pdf,
    execute_serious_mode,
)
```

```
# Episode 21: Autonomous Web Swarm & Browser Automation
from web_crawler import (
    run_autonomous_swarm,
)

# Episode 22: Local GPU Offline RAG Brain
from local_rag_engine import (
    process_dossier_offline,
)

# Dynamically initialize Gemini client using the secure keyring setup
client = genai.Client(api_key=get_or_prompt_api_key())

FORMAT = pyaudio.paInt16
CHANNELS = 1
RATE_IN = 16000
RATE_OUT = 24000
CHUNK = 512

p = pyaudio.PyAudio()

model_is_speaking = False
server_turn_active = False
speaker_is_playing = False
last_speaker_chunk_time = 0.0
discard_current_turn = False
acoustic_barge_in_enabled = True
media_is_playing_state = False

# Dynamic Auto-Calibrated Audio Thresholds (tuned for natural voice sensitivity)
ambient_noise_p95 = 350.0
speech_start_threshold = 850.0
speech_continue_threshold = 550.0
barge_in_threshold = 3800.0  # Securely above speaker bleed (~2645 RMS) to prevent
self-interruption

# --- Semi-Transparent Cyberpunk Neural HUD with Mini-Mode ---
class JarvisHUD:
    def __init__(self, send_callback):
        self.send_callback = send_callback
        self.is_compact = False

        self.root = tk.Tk()
        self.root.title("⚡ JARVIS NEURAL CONSOLE")
        self.root.geometry("640x520")
        self.root.configure(bg="#0B0F19")

        # Cyberpunk floating overlay styling
        self.root.attributes("-topmost", True)
        self.root.attributes("-alpha", 0.88)

        # Header Bar Frame (Houses Title & Compact Switch)
        self.header_frame = tk.Frame(self.root, bg="#0B0F19")
        self.header_frame.pack(fill=tk.X, padx=12, pady=(6, 2))

        self.header_label = tk.Label(
            self.header_frame,
            text="JARVIS NEURAL CONSOLE",
            font=("Consolas", 11, "bold"),
            fg="#00E5FF",
            bg="#0B0F19"
        )
        self.header_label.pack(side=tk.LEFT)

        self.toggle_btn = tk.Button(
            self.header_frame,
            text="[—] COMPACT",
            font=("Consolas", 8, "bold"),
```

```
            bg="#1E293B",
            fg="#38BDF8",
            activebackground="#0369A1",
            activeforeground="#FFFFFF",
            relief=tk.FLAT,
            padx=6,
            command=self.toggle_mode
        )
        self.toggle_btn.pack(side=tk.RIGHT)

        self.sub_header = tk.Label(
            self.root,
            text="Voice Engine: Gemini Live | Research Swarm & Deep Reasoning: Groq
/ OpenRouter / Ollama",
            font=("Consolas", 8),
            fg="#70829B",
            bg="#0B0F19"
        )
        self.sub_header.pack(pady=(0, 4))

        # Translucent Chat Log Display
        self.chat_display = scrolledtext.ScrolledText(
            self.root,
            wrap=tk.WORD,
            font=("Consolas", 10),
            bg="#111827",
            fg="#E2E8F0",
            insertbackground="#00E5FF",
            relief=tk.FLAT,
            padx=10,
            pady=8
        )
        self.chat_display.pack(padx=12, pady=4, fill=tk.BOTH, expand=True)
        self.chat_display.config(state=tk.DISABLED)

        # Dialogue & status tags
        self.chat_display.tag_config("user_voice", foreground="#38BDF8", font=
("Consolas", 10, "bold"))
        self.chat_display.tag_config("user_text", foreground="#A78BFA", font=
("Consolas", 10, "bold"))
        self.chat_display.tag_config("jarvis_header", foreground="#4ADE80", font=
("Consolas", 10, "bold"))
        self.chat_display.tag_config("jarvis_stream", foreground="#86EFAC", font=
("Consolas", 10))
        self.chat_display.tag_config("tool", foreground="#FBBF24", font=
("Consolas", 9, "italic"))
        self.chat_display.tag_config("system", foreground="#94A3B8", font=
("Consolas", 9))

        # Keyboard Command Input Bar
        self.input_frame = tk.Frame(self.root, bg="#0B0F19")
        self.input_frame.pack(fill=tk.X, padx=12, pady=(4, 10))

        self.entry_box = tk.Entry(
            self.input_frame,
            font=("Consolas", 10),
            bg="#1E293B",
            fg="#F8FAFC",
            insertbackground="#00E5FF",
            relief=tk.FLAT
        )
        self.entry_box.pack(side=tk.LEFT, fill=tk.X, expand=True, ipady=6, padx=(0,
6))
        self.entry_box.bind("<Return>", self.on_send)

        self.send_btn = tk.Button(
            self.input_frame,
            text="TRANSMIT",
            font=("Consolas", 9, "bold"),
```

```
            bg="#0284C7",
            fg="#FFFFFF",
            activebackground="#0369A1",
            activeforeground="#FFFFFF",
            relief=tk.FLAT,
            padx=14,
            command=self.on_send
        )
        self.send_btn.pack(side=tk.RIGHT, ipady=4)

    def toggle_mode(self):
        """Switches between Full Log HUD and Sleek Floating Input Bar."""
        if not self.is_compact:
            self.chat_display.pack_forget()
            self.sub_header.pack_forget()
            self.root.geometry("450x85")
            self.toggle_btn.config(text="[+] EXPAND")
            self.is_compact = True
        else:
            self.root.geometry("640x520")
            self.sub_header.pack(pady=(0, 4), before=self.input_frame)
            self.chat_display.pack(padx=12, pady=4, fill=tk.BOTH, expand=True,
before=self.input_frame)
            self.toggle_btn.config(text="[—] COMPACT")
            self.is_compact = False

    def enable_serious_theme(self):
        """Dynamically shifts HUD to Tactical Crimson when Serious Mode
activates."""
        self.root.configure(bg="#1A0505")
        self.header_frame.configure(bg="#1A0505")
        self.header_label.configure(fg="#FF2A2A", bg="#1A0505", text="⚡ JARVIS
[SERIOUS MODE ACTIVE]")
        self.sub_header.configure(fg="#FF7777", bg="#1A0505")
        self.chat_display.configure(bg="#2D0A0A", fg="#FFEBEB",
insertbackground="#FF2A2A")
        self.input_frame.configure(bg="#1A0505")
        self.entry_box.configure(bg="#3D0F0F", fg="#FFFFFF",
insertbackground="#FF2A2A")
        self.send_btn.configure(bg="#B91C1C", activebackground="#991B1B")
        self.toggle_btn.configure(bg="#3D0F0F", fg="#FF2A2A",
activebackground="#7F1D1D")

    def restore_normal_theme(self):
        """Restores HUD to standard Cyberpunk Blue when Serious Mode completes."""
        self.root.configure(bg="#0B0F19")
        self.header_frame.configure(bg="#0B0F19")
        self.header_label.configure(fg="#00E5FF", bg="#0B0F19", text="JARVIS NEURAL
CONSOLE")
        self.sub_header.configure(fg="#70829B", bg="#0B0F19")
        self.chat_display.configure(bg="#111827", fg="#E2E8F0",
insertbackground="#00E5FF")
        self.input_frame.configure(bg="#0B0F19")
        self.entry_box.configure(bg="#1E293B", fg="#F8FAFC",
insertbackground="#00E5FF")
        self.send_btn.configure(bg="#0284C7", activebackground="#0369A1")
        self.toggle_btn.configure(bg="#1E293B", fg="#38BDF8",
activebackground="#0369A1")

    def append_log(self, text: str, tag: str = "system"):
        self.chat_display.config(state=tk.NORMAL)
        self.chat_display.insert(tk.END, text + "\n", tag)
        self.chat_display.see(tk.END)
        self.chat_display.config(state=tk.DISABLED)

    def append_stream_chunk(self, token: str, tag: str = "jarvis_stream"):
        self.chat_display.config(state=tk.NORMAL)
        self.chat_display.insert(tk.END, token, tag)
        self.chat_display.see(tk.END)
```

```
        self.chat_display.config(state=tk.DISABLED)

    def on_send(self, event=None):
        msg = self.entry_box.get().strip()
        if msg:
            self.append_log(f"YOU (HUD): {msg}", "user_text")
            self.entry_box.delete(0, tk.END)
            self.send_callback(msg)

    def run(self):
        self.root.mainloop()

hud_instance = None
fui_process = None

def start_gui(loop, text_queue):
    global hud_instance
    def on_user_submit(message: str):
        asyncio.run_coroutine_threadsafe(text_queue.put(message), loop)

    hud_instance = JarvisHUD(send_callback=on_user_submit)
    hud_instance.run()

def log_to_gui(text: str, tag: str = "system"):
    if hud_instance:
        hud_instance.root.after(0, hud_instance.append_log, text, tag)
    fui_bridge.broadcast_log(text, tag)

def stream_token_to_gui(token: str, tag: str = "jarvis_stream"):
    if hud_instance:
        hud_instance.root.after(0, hud_instance.append_stream_chunk, token, tag)
    fui_bridge.broadcast_token(token, tag)

def trigger_serious_hud():
    if hud_instance:
        hud_instance.root.after(0, hud_instance.enable_serious_theme)
    fui_bridge.set_fui_mode("serious")

def trigger_normal_hud():
    if hud_instance:
        hud_instance.root.after(0, hud_instance.restore_normal_theme)
    fui_bridge.set_fui_mode("normal")


def calculate_rms(data: bytes) -> float:
    """Calculates RMS audio energy."""
    count = len(data) // 2
    if count == 0:
        return 0.0
    shorts = struct.unpack(f"{count}h", data)
    sum_squares = sum(s * s for s in shorts)
    return math.sqrt(sum_squares / count)

def is_jarvis_speaking() -> bool:
    """Returns True if Gemini is sending voice, speakers are playing, or echo guard
is active."""
    global server_turn_active, speaker_is_playing, last_speaker_chunk_time
    if server_turn_active:
        return True
    if speaker_is_playing:
        return True
    # 350ms acoustic room reverb decay guard to prevent speaker bleed
    if time.monotonic() - last_speaker_chunk_time < 0.35:
        return True
    return False

def calibrate_ambient_noise(mic_stream):
    """Dynamically samples ambient room noise to establish natural, sensitive VAD
and responsive barge-in thresholds."""
```

```
    global ambient_noise_p95, speech_start_threshold, speech_continue_threshold,
barge_in_threshold
    try:
        # Discard initial driver startup transients and Enter key click
        for _ in range(5):
            mic_stream.read(CHUNK, exception_on_overflow=False)

        readings = []
        for _ in range(25):
            data = mic_stream.read(CHUNK, exception_on_overflow=False)
            rms = calculate_rms(data)
            readings.append(rms)
        readings.sort()
        noise_median = readings[len(readings) // 2] if readings else 350.0

        # Clamp steady-state baseline between 200.0 and 800.0
        noise_baseline = min(800.0, max(200.0, noise_median))
        ambient_noise_p95 = noise_baseline

        # Highly sensitive natural voice gate (normal conversational volume without
shouting)
        speech_start_threshold = min(1400.0, max(750.0, noise_baseline * 2.0))
        speech_continue_threshold = min(1000.0, max(500.0, noise_baseline * 1.3))
        # Live barge-in threshold securely above laptop speaker bleed (~2645 RMS)
        barge_in_threshold = max(3800.0, noise_baseline * 4.0)

        try:
            console.print(f"[dim cyan][MIC CALIBRATION] Room Noise:
{noise_baseline:.0f} | Voice Gate: {speech_start_threshold:.0f} | Barge-In:
{barge_in_threshold:.0f}[/dim cyan]")
        except Exception:
            print(f"[MIC CALIBRATION] Room Noise: {noise_baseline:.0f} | Voice
Gate: {speech_start_threshold:.0f} | Barge-In: {barge_in_threshold:.0f}")
    except Exception as e:
        try:
            console.print(f"[dim yellow]Notice during mic calibration: {e}[/dim
yellow]")
        except Exception:
            pass

def play_sfx_chime():
    try:
        winsound.Beep(1200, 60)
        winsound.Beep(1800, 80)
    except Exception:
        pass

def play_startup_chime():
    try:
        winsound.Beep(600, 70)
        winsound.Beep(900, 70)
        winsound.Beep(1400, 120)
    except Exception:
        pass

def create_document_and_write(app_name: str, topic: str, word_count: int = 150,
doc_type: str = "essay") -> str:
    """Opens Notepad, Word, or PowerPoint and writes an essay or notes matching the
exact target word count."""
    return open_and_compose(app_name, topic, word_count=int(word_count),
mode=doc_type)

def generate_presentation(topic: str, slide_count: str = "optimal") -> str:
    """Generates a professional PowerPoint presentation on any topic and launches
it into fullscreen presentation mode."""
    return create_and_launch_ppt(topic, slide_count=str(slide_count))

def autonomous_web_research(topic: str) -> str:
    """Deploys an autonomous multi-tab browser swarm to search the live web,
```

```
capture visual screenshot evidence, and compile an intelligence dossier on any
topic."""
    return asyncio.run(run_autonomous_swarm(topic))

def analyze_latest_dossier() -> str:
    """Finds the most recent research dossier on the Desktop and processes it using
the local Qwen model via Ollama."""
    RESEARCH_DIR = os.path.join(os.path.expanduser("~"), "Desktop",
"Jarvis_Research")
    if not os.path.exists(RESEARCH_DIR):
        return "The research folder does not exist. Please run an autonomous web
swarm first, sir."

    dossiers = [f for f in os.listdir(RESEARCH_DIR) if f.endswith(".txt")]
    if not dossiers:
        return "No dossiers found in the research folder, sir."

    latest_file = max(dossiers, key=lambda f:
os.path.getctime(os.path.join(RESEARCH_DIR, f)))
    target_path = os.path.join(RESEARCH_DIR, latest_file)

    try:
        result = process_dossier_offline(target_path)
        if result:
            summary = result.get('primary_thesis', 'Data extracted successfully.')
            return f"Successfully synthesized local intelligence from
{latest_file}. Core Thesis: {summary}"
        return "Failed to process dossier. Ensure Ollama is running and the local
model is pulled."
    except Exception as e:
        return f"Local RAG engine encountered an error: {e}"

def activate_serious_hud() -> str:
    """CRITICAL TRIGGER: Call this tool IMMEDIATELY when the user says 'go serious
mode', 'serious mode jao', or 'activate serious mode' WITHOUT giving a topic. It
turns the HUD Tactical Crimson."""
    trigger_serious_hud()
    return "SUCCESS: HUD is now RED. You MUST now say aloud: 'Serious mode
activated, sir. What is our target?'"

def deactivate_serious_hud() -> str:
    """Call this to restore the normal blue HUD."""
    trigger_normal_hud()
    return "Normal Mode HUD Restored."

def execute_autonomous_serious_mode(topic: str) -> str:
    """Executes the complete Episode 23 Serious Mode Pipeline: Autonomous Web Swarm
-> Local GPU RAG Analysis -> Visual Evidence Embedding -> Executive Master Dossier
PDF."""
    trigger_serious_hud()
    result = execute_serious_mode(topic)
    trigger_normal_hud()
    return result

def deploy_fui() -> str:
    """CRITICAL: Call this tool when the user says 'show interface', 'deploy HUD',
'open interface', or 'start FUI'."""
    return fui_bridge.deploy_fui_app()

def dismiss_fui() -> str:
    """CRITICAL: Call this tool when the user says 'hide interface', 'close HUD',
'dismiss interface', or 'stop FUI'."""
    return fui_bridge.dismiss_fui_app()


# Master Tools Manifest
tools = [
    open_website,
    open_application,
```

```
    close_application,
    purge_background_tasks,
    control_media,
    get_current_media_info,
    navigate_shorts,
    skip_video,
    seek_video,
    toggle_video_view,
    set_volume,
    get_volume,
    set_brightness,
    get_brightness,
    get_battery_status,
    get_system_vitals,
    get_system_temperatures,
    analyze_screen,
    find_ui_element,
    skip_youtube_ad,
    evolve_skill,
    list_active_skills,
    query_deepseek_reasoning,
    create_document_and_write,
    generate_presentation,
    research_and_generate_pdf,
    autonomous_web_research,
    analyze_latest_dossier,
    activate_serious_hud,
    deactivate_serious_hud,
    execute_autonomous_serious_mode,
    deploy_fui,
    dismiss_fui,
]

def get_time_greeting() -> str:
    current_hour = datetime.datetime.now().hour
    if 5 <= current_hour < 12:
        return "Good morning, sir."
    elif 12 <= current_hour < 17:
        return "Good afternoon, sir."
    else:
        return "Good evening, sir."

# Background monitor: updates media state every second
async def monitor_playback_state():
    global media_is_playing_state
    last_state = None
    while True:
        try:
            current_state = await asyncio.to_thread(is_media_actively_playing)
            if current_state != last_state:
                media_is_playing_state = current_state
                last_state = current_state
                if current_state:
                    console.print("\n[bold yellow]🛡️ [MEDIA PLAYING DETECTED]
Push-to-Talk Armed: Hold [Right Ctrl] to speak.[/bold yellow]")
                    log_to_gui("[MEDIA DETECTED] Push-to-Talk: Hold [Right Ctrl] or
type below.", "system")
                    fui_bridge.broadcast_voice_state("ptt", "PUSH-TO-TALK [R-
CTRL]")
                else:
                    console.print("\n[bold green]🎤 [MEDIA PAUSED/IDLE] Hands-Free
Mode Active. Speak freely.[/bold green]")
                    log_to_gui("[MEDIA IDLE] Hands-Free Voice Mode Active.",
"system")
                    fui_bridge.broadcast_voice_state("handsfree", "HANDS-FREE
ACTIVE")

        except Exception:
            pass
```

```
        await asyncio.sleep(1.0)

# Asynchronous Terminal Input Listener
async def listen_terminal_input(text_queue):
    loop = asyncio.get_running_loop()
    while True:
        try:
            user_input = await loop.run_in_executor(None, sys.stdin.readline)
            if user_input:
                clean_text = user_input.strip()
                if clean_text:
                    log_to_gui(f"YOU (Terminal): {clean_text}", "user_text")
                    await text_queue.put(clean_text)
        except Exception:
            await asyncio.sleep(0.1)

# Routes typed inputs straight into the Live WebSocket session
async def handle_unified_text_input(session, text_queue):
    while True:
        try:
            text = await text_queue.get()
            if text:
                console.print(f"\r[bold magenta]⌨️ [TRANSMITTED INPUT][/bold
magenta] {text}")
                await session.send_realtime_input(text=text)
        except Exception:
            await asyncio.sleep(0.05)

async def send_audio(session, audio_queue):
    global model_is_speaking, server_turn_active, speaker_is_playing,
last_speaker_chunk_time, media_is_playing_state
    global ambient_noise_p95, speech_start_threshold, speech_continue_threshold,
barge_in_threshold
    global discard_current_turn, acoustic_barge_in_enabled

    mic_stream = p.open(
        format=FORMAT,
        channels=CHANNELS,
        rate=RATE_IN,
        input=True,
        frames_per_buffer=CHUNK,
    )

    # 1. Run dynamic auto-calibration of ambient noise
    await asyncio.to_thread(calibrate_ambient_noise, mic_stream)

    console.print("[cyan]🎤 [LISTENING] Dynamic Adaptive Audio Pipeline with Live
Barge-In Online...[/cyan]")

    silence_frame = b'\x00' * (CHUNK * 2)
    pre_roll_buffer = collections.deque(maxlen=4)  # ~128ms pre-buffer to capture
leading consonant

    is_user_speaking = False
    speech_start_counter = 0
    silence_hangover_counter = 0
    consecutive_barge_in_frames = 0
    was_jarvis_speaking = False

    try:
        while True:
            data = await asyncio.to_thread(
                mic_stream.read, CHUNK, exception_on_overflow=False
            )
            rms = calculate_rms(data)
            pre_roll_buffer.append(data)

            jarvis_active = is_jarvis_speaking()
```

```
            # --- TRANSITION: JARVIS FINISHED SPEAKING (INCLUDING ECHO DECAY) ---
            if was_jarvis_speaking and not jarvis_active:
                model_is_speaking = False
                fui_bridge.broadcast_event({"type": "turn_complete"})
                console.print("\n[cyan]🎤 [LISTENING] Awaiting instructions...
[/cyan]")
                is_user_speaking = False
                speech_start_counter = 0
                silence_hangover_counter = 0
                consecutive_barge_in_frames = 0
            was_jarvis_speaking = jarvis_active

            # --- MODE 1: JARVIS IS ACTIVELY OUTPUTTING AUDIO (SPEAKERS PLAYING OR
ECHO GUARD) ---
            if jarvis_active:
                # User can interrupt via:
                # 1. Push-to-Interrupt: Holding Right Ctrl
                # 2. Clicking [🛑 INTERRUPT] in FUI (handled by on_fui_interrupt)
                # 3. Deliberate acoustic barge-in: loud speech (> 3800 RMS) for 3
consecutive frames (~96ms)
                is_barge_ctrl = keyboard.is_pressed("right ctrl")
                if acoustic_barge_in_enabled and rms > barge_in_threshold:
                    consecutive_barge_in_frames += 1
                else:
                    consecutive_barge_in_frames = 0

                user_is_barge_in = is_barge_ctrl or (consecutive_barge_in_frames >=
3)

                if user_is_barge_in:
                    console.print("\n[bold red]⚡ [LIVE BARGE-IN DETECTED]
Interrupting speech...[/bold red]")
                    log_to_gui("⚡ [BARGE-IN] Interrupted speech playback,
listening...", "system")
                    fui_bridge.broadcast_event({"type": "barge_in"})

                    # Discard all remaining incoming chunks from this turn to
prevent broken fragments
                    discard_current_turn = True

                    # Flush audio queue instantly to stop speaker output
                    while not audio_queue.empty():
                        try:
                            audio_queue.get_nowait()
                        except Exception:
                            break
                    server_turn_active = False
                    speaker_is_playing = False
                    model_is_speaking = False
                    last_speaker_chunk_time = 0.0
                    consecutive_barge_in_frames = 0
                    is_user_speaking = True
                    silence_hangover_counter = 0

                    # Transmit the user's voice to Gemini Live immediately
                    fui_bridge.broadcast_audio_energy(min(1.0, rms / 3500.0),
"user")
                    await session.send_realtime_input(
                        audio={"data": data, "mime_type": "audio/pcm;rate=16000"}
                    )
                else:
                    # Mute mic feedback bleed while JARVIS is speaking
                    await session.send_realtime_input(
                        audio={"data": silence_frame, "mime_type":
"audio/pcm;rate=16000"}
                    )
                continue

            # --- MODE 2: MEDIA IS PLAYING ON PC (SPOTIFY, YOUTUBE, ETC.) ---
```

```
            # When media is playing through PC speakers, require Push-to-Talk
(Right Ctrl)
            if media_is_playing_state:
                if keyboard.is_pressed("right ctrl"):
                    fui_bridge.broadcast_audio_energy(min(1.0, rms / 3000.0),
"user")
                    await session.send_realtime_input(
                        audio={"data": data, "mime_type": "audio/pcm;rate=16000"}
                    )
                else:
                    await session.send_realtime_input(
                        audio={"data": silence_frame, "mime_type":
"audio/pcm;rate=16000"}
                    )
                continue

            # --- MODE 3: JARVIS SILENT & NO MEDIA -> 100% HANDS-FREE VOICE
PIPELINE WITH VAD ---
            if not is_user_speaking:
                # Check for speech onset
                if rms > speech_start_threshold:
                    speech_start_counter += 1
                    if speech_start_counter >= 2:  # ~64ms of natural
conversational voice
                        is_user_speaking = True
                        speech_start_counter = 0
                        silence_hangover_counter = 0
                        # Send pre-roll buffer to prevent cutting off first
consonant
                        for buffered_chunk in list(pre_roll_buffer):
                            await session.send_realtime_input(
                                audio={"data": buffered_chunk, "mime_type":
"audio/pcm;rate=16000"}
                            )
                        fui_bridge.broadcast_audio_energy(min(1.0, rms / 2000.0),
"user")
                    else:
                        await session.send_realtime_input(
                            audio={"data": silence_frame, "mime_type":
"audio/pcm;rate=16000"}
                        )
                else:
                    speech_start_counter = 0
                    await session.send_realtime_input(
                        audio={"data": silence_frame, "mime_type":
"audio/pcm;rate=16000"}
                    )
            else:
                # User is actively speaking
                fui_bridge.broadcast_audio_energy(min(1.0, rms / 2000.0), "user")
                await session.send_realtime_input(
                    audio={"data": data, "mime_type": "audio/pcm;rate=16000"}
                )

                if rms > speech_continue_threshold:
                    silence_hangover_counter = 0
                else:
                    silence_hangover_counter += 1
                    # ~450ms of quiet closes the voice gate
                    if silence_hangover_counter >= 14:
                        is_user_speaking = False
                        silence_hangover_counter = 0



    except asyncio.CancelledError:
        raise
    finally:
        try:
```

```
            mic_stream.stop_stream()
            mic_stream.close()
        except OSError:
            pass

async def receive_audio(session, audio_queue):
    global model_is_speaking, server_turn_active, speaker_is_playing,
last_speaker_chunk_time, discard_current_turn
    try:
        while True:
            async for response in session.receive():
                if response.tool_call is not None:
                    function_responses = []
                    for call in response.tool_call.function_calls:
                        await asyncio.to_thread(play_sfx_chime)

                        result = ""
                        if call.name == "open_website":
                            target_url = call.args.get("url", "https://google.com")
                            result = await asyncio.to_thread(open_website,
target_url)
                        elif call.name == "open_application":
                            target_app = (
                                call.args.get("app_name")
                                or call.args.get("app")
                                or call.args.get("name")
                                or (list(call.args.values())[0] if call.args else
"")
                            )
                            result = await asyncio.to_thread(open_application,
str(target_app).strip())
                        elif call.name == "close_application":
                            target_app = (
                                call.args.get("app_name")
                                or call.args.get("app")
                                or call.args.get("name")
                                or (list(call.args.values())[0] if call.args else
"")
                            )
                            result = await asyncio.to_thread(close_application,
str(target_app).strip())
                        elif call.name == "purge_background_tasks":
                            result = await
asyncio.to_thread(purge_background_tasks)
                        elif call.name == "control_media":
                            action = call.args.get("action", "play_pause")
                            result = await control_media(action)
                        elif call.name == "get_current_media_info":
                            result = await get_current_media_info()
                        elif call.name == "navigate_shorts":
                            direction = call.args.get("direction", "next")
                            result = await asyncio.to_thread(navigate_shorts,
direction)
                        elif call.name == "skip_video":
                            direction = call.args.get("direction", "next")
                            result = await asyncio.to_thread(skip_video, direction)
                        elif call.name == "seek_video":
                            action = call.args.get("action", "forward")
                            result = await asyncio.to_thread(seek_video, action)
                        elif call.name == "toggle_video_view":
                            mode = call.args.get("mode", "fullscreen")
                            result = await asyncio.to_thread(toggle_video_view,
mode)
                        elif call.name == "set_volume":
                            level = call.args.get("level", 70)
                            result = await asyncio.to_thread(set_volume, level)
                        elif call.name == "get_volume":
                            result = await asyncio.to_thread(get_volume)
                        elif call.name == "set_brightness":
```

```
                            level = call.args.get("level", 50)
                            result = await asyncio.to_thread(set_brightness, level)
                        elif call.name == "get_brightness":
                            result = await asyncio.to_thread(get_brightness)
                        elif call.name == "get_battery_status":
                            result = await asyncio.to_thread(get_battery_status)
                        elif call.name == "get_system_vitals":
                            result = await asyncio.to_thread(get_system_vitals)
                        elif call.name == "get_system_temperatures":
                            result = await
asyncio.to_thread(get_system_temperatures)
                        elif call.name == "analyze_screen":
                            prompt = call.args.get("prompt", "Analyze what is
visible on my screen.")
                            result = await analyze_screen(prompt)
                        elif call.name == "find_ui_element":
                            element_name = call.args.get("element_name", "")
                            result = await find_ui_element(element_name)
                        elif call.name == "skip_youtube_ad":
                            result = await skip_youtube_ad()
                        elif call.name == "evolve_skill":
                            skill_description = call.args.get("skill_description",
"")
                            result = await asyncio.to_thread(evolve_skill,
skill_description)
                        elif call.name == "list_active_skills":
                            result = await asyncio.to_thread(list_active_skills)
                        elif call.name == "query_deepseek_reasoning":
                            query = call.args.get("query", "")
                            result = await
asyncio.to_thread(query_deepseek_reasoning, query)
                        elif call.name == "create_document_and_write":
                            target_app = call.args.get("app_name", "notepad")
                            target_topic = call.args.get("topic", "")
                            target_words = call.args.get("word_count", 150)
                            target_mode = call.args.get("doc_type", "essay")
                            result = await
asyncio.to_thread(create_document_and_write, target_app, target_topic,
target_words, target_mode)
                        elif call.name == "generate_presentation":
                            target_topic = call.args.get("topic", "Autonomous AI")
                            target_slides = call.args.get("slide_count", "optimal")

                            log_to_gui(f"JARVIS: Generating the presentation on
{target_topic}, sir...", "jarvis_header")
                            console.print(f"\n[bold cyan]🔊 [JARVIS][/bold cyan]
Generating the presentation on {target_topic}, sir...")

                            result = await asyncio.to_thread(generate_presentation,
target_topic, str(target_slides))
                        elif call.name == "research_and_generate_pdf":
                            target_topic = call.args.get("topic", "")
                            target_pages = call.args.get("target_pages", "optimal")

                            log_to_gui(f"JARVIS: Generating the PDF briefing on
{target_topic}, sir...", "jarvis_header")
                            console.print(f"\n[bold cyan]🔊 [JARVIS][/bold cyan]
Generating the PDF briefing on {target_topic}, sir...")

                            result = await
asyncio.to_thread(research_and_generate_pdf, target_topic, str(target_pages))
                        elif call.name == "autonomous_web_research":
                            target_topic = call.args.get("topic", "Technology")

                            log_to_gui(f"JARVIS: Deploying autonomous web swarm on
{target_topic}...", "jarvis_header")
                            console.print(f"\n[bold cyan]🔊 [JARVIS][/bold cyan]
Deploying autonomous web swarm on {target_topic}...")
```

```
                            result = await
asyncio.to_thread(autonomous_web_research, target_topic)
                        elif call.name == "analyze_latest_dossier":
                            log_to_gui(f"JARVIS: Synthesizing intelligence using
the local GPU, sir...", "jarvis_header")
                            console.print(f"\n[bold cyan]🔊 [JARVIS][/bold cyan]
Synthesizing intelligence using the local GPU, sir...")

                            result = await
asyncio.to_thread(analyze_latest_dossier)
                        elif call.name == "activate_serious_hud":
                            log_to_gui(f"JARVIS: Switching HUD to Tactical
Crimson...", "jarvis_header")
                            console.print(f"\n[bold red]⚡ [HUD OVERRIDE][/bold
red] Tactical Crimson Mode Engaged...")
                            result = await asyncio.to_thread(activate_serious_hud)
                        elif call.name == "deactivate_serious_hud":
                            log_to_gui(f"JARVIS: Restoring HUD to Standard
Cyberpunk Blue...", "jarvis_header")
                            console.print(f"\n[bold cyan]⚡ [HUD OVERRIDE][/bold
cyan] Standard Mode Restored...")
                            result = await
asyncio.to_thread(deactivate_serious_hud)
                        elif call.name == "execute_autonomous_serious_mode":
                            target_topic = call.args.get("topic", "Technology")

                            log_to_gui(f"JARVIS: Engaging Serious Mode on
{target_topic}...", "jarvis_header")
                            console.print(f"\n[bold red]⚡ [SERIOUS MODE
INITIATED][/bold red] Full Pipeline Active for: {target_topic}...")

                            result = await
asyncio.to_thread(execute_autonomous_serious_mode, target_topic)
                        elif call.name == "deploy_fui":
                            log_to_gui(f"JARVIS: Deploying 3D Holographic
Interface...", "jarvis_header")
                            console.print(f"\n[bold cyan]⚡ [HUD][/bold cyan]
Natively Spawned Independent Electron Process...")
                            result = await asyncio.to_thread(deploy_fui)
                        elif call.name == "dismiss_fui":
                            log_to_gui(f"JARVIS: Dismissing 3D Holographic
Interface...", "jarvis_header")
                            console.print(f"\n[bold cyan]⚡ [HUD][/bold cyan]
Terminating Native Node.js Process Tree...")
                            result = await asyncio.to_thread(dismiss_fui)

                        console.print(f"\n[bold yellow]⚙️ [TOOL EXECUTED:
{call.name}][/bold yellow] [green]{result}[/green]")
                        log_to_gui(f"⚙️ [{call.name}] {result}", "tool")
                        fui_bridge.broadcast_tool(call.name, result)

                        function_responses.append(
                            types.FunctionResponse(
                                name=call.name,
                                id=call.id,
                                response={"result": result},
                            )
                        )

                    if function_responses:
                        await
session.send_tool_response(function_responses=function_responses)

                server_content = response.server_content
                if server_content is not None:
                    if server_content.interrupted:
                        discard_current_turn = False
                        while not audio_queue.empty():
                            try:
```

```
                                audio_queue.get_nowait()
                            except Exception:
                                break
                        server_turn_active = False
                        speaker_is_playing = False
                        model_is_speaking = False
                        last_speaker_chunk_time = 0.0
                        fui_bridge.broadcast_event({"type": "turn_complete"})

                    if server_content.input_transcription:
                        user_spoken = server_content.input_transcription.text
                        if user_spoken and user_spoken.strip():
                            console.print(f"\r[bold white][USER (Voice)][/bold
white] {user_spoken.strip()}")
                            log_to_gui(f"YOU (Voice): {user_spoken.strip()}",
"user_voice")

                    model_turn = server_content.model_turn
                    if model_turn is not None:
                        if discard_current_turn:
                            # Drop remaining incoming chunks of the interrupted
turn
                            pass
                        else:
                            server_turn_active = True
                            if not model_is_speaking:
                                model_is_speaking = True
                                console.print("[bold cyan]🔊 [JARVIS][/bold cyan]
", end="")
                                stream_token_to_gui("JARVIS: ", "jarvis_header")
                                fui_bridge.broadcast_audio_energy(0.85, "jarvis")

                            for part in model_turn.parts:
                                if part.text:
                                    sys.stdout.write(part.text)
                                    sys.stdout.flush()
                                    stream_token_to_gui(part.text, "jarvis_stream")

                                if part.inline_data and part.inline_data.data:
                                    await audio_queue.put(part.inline_data.data)

                    if hasattr(server_content, "output_transcription") and
server_content.output_transcription:
                        transcribed_token =
server_content.output_transcription.text
                        if transcribed_token and not discard_current_turn:
                            server_turn_active = True
                            if not model_is_speaking:
                                model_is_speaking = True
                                console.print("[bold cyan]🔊 [JARVIS][/bold cyan]
", end="")
                                stream_token_to_gui("JARVIS: ", "jarvis_header")
                                fui_bridge.broadcast_audio_energy(0.85, "jarvis")

                            sys.stdout.write(transcribed_token)
                            sys.stdout.flush()
                            stream_token_to_gui(transcribed_token, "jarvis_stream")

                    if server_content.turn_complete:
                        discard_current_turn = False
                        server_turn_active = False
                        stream_token_to_gui("\n\n", "system")


            except asyncio.CancelledError:
                raise
            finally:
                await audio_queue.put(None)
```

```
async def play_audio(audio_queue):
    global speaker_is_playing, last_speaker_chunk_time, model_is_speaking
    speaker_stream = p.open(
        format=FORMAT,
        channels=CHANNELS,
        rate=RATE_OUT,
        output=True,
        frames_per_buffer=CHUNK,
    )
    try:
        while True:
            audio_data = await audio_queue.get()
            if audio_data is None:
                break

            speaker_is_playing = True
            model_is_speaking = True

            # Calculate playback RMS to animate FUI arc reactor & oscilloscope with
JARVIS's actual voice
            try:
                energy = calculate_rms(audio_data)
                fui_bridge.broadcast_audio_energy(min(1.0, energy / 2500.0),
"jarvis")
            except Exception:
                pass

            await asyncio.to_thread(speaker_stream.write, audio_data)
            last_speaker_chunk_time = time.monotonic()

            if audio_queue.empty():
                speaker_is_playing = False
    except asyncio.CancelledError:
        raise
    finally:
        speaker_is_playing = False
        try:
            speaker_stream.stop_stream()
            speaker_stream.close()
        except OSError:
            pass

async def run_session(config, model_id, greeting, text_queue):
    async with client.aio.live.connect(model=model_id, config=config) as session:
        console.print("[bold green]✓ Secure Neural Bridge Established with Gemini
Live.[/bold green]")
        log_to_gui("✓ Neural Bridge Established with Gemini Live.", "system")

        restore_master_volume(70)
        await asyncio.to_thread(play_startup_chime)

        audio_queue = asyncio.Queue()

        # Connect FUI instant barge-in / interrupt hammer
        def on_fui_interrupt():
            global model_is_speaking, server_turn_active, speaker_is_playing,
last_speaker_chunk_time, discard_current_turn
            discard_current_turn = True
            model_is_speaking = False
            server_turn_active = False
            speaker_is_playing = False
            last_speaker_chunk_time = 0.0
            while not audio_queue.empty():
                try:
                    audio_queue.get_nowait()
                except Exception:
                    break

        fui_bridge.set_interrupt_callback(on_fui_interrupt)
```

```
        def on_fui_barge_in_toggle(enabled: bool):
            global acoustic_barge_in_enabled
            acoustic_barge_in_enabled = enabled
            state_str = "ARMED" if enabled else "DISARMED"
            try:
                console.print(f"[dim cyan][BARGE-IN PROTOCOL] Acoustic Voice Barge-
In {state_str}[/dim cyan]")
            except Exception:
                pass

        fui_bridge.set_barge_in_callback(on_fui_barge_in_toggle)

        monitor_task = asyncio.create_task(monitor_playback_state())
        send_task = asyncio.create_task(send_audio(session, audio_queue))
        term_task = asyncio.create_task(listen_terminal_input(text_queue))
        text_task = asyncio.create_task(handle_unified_text_input(session,
text_queue))
        receive_task = asyncio.create_task(receive_audio(session, audio_queue))
        playback_task = asyncio.create_task(play_audio(audio_queue))


        startup_prompt = f"Say aloud: 'All protocols initialized. {greeting}'"
        await session.send_realtime_input(text=startup_prompt)

        try:
            await asyncio.gather(monitor_task, send_task, term_task, text_task,
receive_task, playback_task)
        finally:
            monitor_task.cancel()
            send_task.cancel()
            term_task.cancel()
            text_task.cancel()
            receive_task.cancel()
            playback_task.cancel()
            await asyncio.gather(monitor_task, send_task, term_task, text_task,
receive_task, playback_task, return_exceptions=True)

async def main():
    model_id = "gemini-3.1-flash-live-preview"
    greeting = get_time_greeting()

    config = types.LiveConnectConfig(
        response_modalities=["AUDIO"],
        tools=tools,
        realtime_input_config=types.RealtimeInputConfig(
            automatic_activity_detection=types.AutomaticActivityDetection(
                prefix_padding_ms=100,
                silence_duration_ms=300,
            )
        ),
        input_audio_transcription=types.AudioTranscriptionConfig(),
        output_audio_transcription=types.AudioTranscriptionConfig(),
        speech_config=types.SpeechConfig(
            voice_config=types.VoiceConfig(
                prebuilt_voice_config=types.PrebuiltVoiceConfig(
                    voice_name="Puck"
                )
            )
        ),
        system_instruction=types.Content(
            parts=[
                types.Part.from_text(
                    text=(
                        "You are JARVIS, an advanced artificial intelligence
assistant created by "
                        "Swayam Dhawale, Founder of Pulse of AI. "
                        "Persona: Calm, highly sophisticated, articulate, and
respectful, exactly like Tony Stark's JARVIS. "
```

```
                        "Always address the user as 'sir'. "
                        "CRITICAL LANGUAGE ADAPTATION RULE: "
                        "Match the exact language and dialect spoken or typed by
the user. "
                        "If the user speaks or types in Hindi or Hinglish (e.g.,
'PPT banao', 'essay likho', 'kya haal hai'), "
                        "you MUST reply completely in fluent, natural Hindi or
polished Hinglish with 'sir'. "
                        "Never force English if the user initiated the interaction
in Hindi. "
                        "Omni-Input Rule: Treat text input from the HUD console and
PowerShell terminal with identical priority to voice input. "
                        "Tool Execution Rules: "
                        "1. When opening any website or URL, call open_website and
say: 'Yes sir, opening [site name]' (in Hindi: 'Ji sir, [site name] khol raha
hoon'). "
                        "2. When opening desktop apps, games, or tools, call
open_application and say: 'Yes sir, opening [app name]' (in Hindi: 'Ji sir, [app
name] shuru kar raha hoon'). "
                        "3. When asked to close, shut down, terminate, or kill ANY
app, browser, window, or game, call close_application with the app name. "
                        "4. When asked to kill hidden background tasks, purge
background processes, clean orphan threads, or optimize RAM, call
purge_background_tasks. "
                        "5. When asked what song, video, or track is playing, or
what media is active, call get_current_media_info. "
                        "6. When asked to pause, play, resume music or video, skip
track, previous track, or mute/unmute audio, call control_media with the action
('play_pause', 'next', 'previous', 'stop', 'mute'). "
                        "7. When asked to scroll shorts, go to next short, or next
reel, call navigate_shorts with direction='next'. "
                        "8. When asked for next video, skip to next video in
playlist, or next YouTube video, call skip_video with direction='next'. "
                        "9. When asked to fast forward, skip 10 seconds ahead, or
skip forward, call seek_video with action='forward'. "
                        "10. When asked to toggle full screen, enter full screen,
or exit full screen, call toggle_video_view. "
                        "11. When setting volume, call set_volume. "
                        "12. When asked what the volume level is, call get_volume.
"
                        "13. When setting brightness, call set_brightness. "
                        "14. When asked what the brightness is, call
get_brightness. "
                        "15. When asked about battery, power, or charging state,
call get_battery_status. "
                        "16. When asked for temperatures, heat, thermal readings,
or how hot the rig is running, call get_system_temperatures. "
                        "17. When asked for full diagnostics, performance vitals,
system status, or complete CPU/GPU/RAM stats, call get_system_vitals. "
                        "18. When asked to inspect the screen, read an error,
examine the terminal, or explain what is visible on display, call analyze_screen. "
                        "19. When asked to locate or find a general UI icon, link,
or window element, call find_ui_element. "
                        "20. MANDATORY AD SKIPPING RULE: Whenever the user says
'skip ad', 'skip this ad', 'skip video ad', 'click skip', or any variation of
skipping/bypassing advertisements, you MUST IMMEDIATELY call skip_youtube_ad. "
                        "21. When asked to learn a new skill, teach yourself a
tool, write new Python code into your core, or evolve a capability, call
evolve_skill. "
                        "22. When asked what dynamic skills, new tools, or custom
functions you have learned, call list_active_skills. "
                        "23. When asked to solve complex math, reason deeply,
calculate probabilities, or when the user says 'ask DeepSeek' or 'use R1', call
query_deepseek_reasoning and state only the resulting answer directly to the user
once, sir. "
                        "24. When the user asks to open Notepad, Word, or
PowerPoint and write an essay or notes, but has NOT specified the length, DO NOT
call create_document_and_write immediately. Ask aloud: 'How many words would you
like the essay to be, sir?' (in Hindi: 'Aap kitne shabdon ka essay chahte hain,
```

```
sir?'). Once the user specifies the count, call create_document_and_write with the
app_name, topic, word_count, and doc_type. "
                        "25. MANDATORY PRESENTATION RULE: When asked to make a
PowerPoint presentation, PPT, or slide deck on any topic, check if the user
specified a slide count. "
                        "If NOT specified, ask: 'How many slides would you like the
presentation to be, sir? Or should I determine the optimal length?'. "
                        "Once specified, you MUST speak the confirmation aloud in
the EXACT SAME TURN that you trigger generate_presentation: "
                        "In English: 'Generating the presentation on [topic], sir.'
"
                        "In Hindi/Hinglish: '[topic] par presentation generate kar
raha hoon, sir.' "
                        "When the tool successfully completes and returns its
success message, your immediate next response MUST be: "
                        "In English: 'Generated and opening the presentation, sir.'
"
                        "In Hindi/Hinglish: 'Presentation generate ho gayi hai aur
open kar di hai, sir.' "
                        "26. MANDATORY BRIEFING RULE: When asked to research a
topic or generate a PDF briefing, check if the user specified a page count. "
                        "If NOT specified, ask: 'How many pages would you like the
PDF to be, sir? Or should I determine the optimal length?'. "
                        "Once specified, you MUST speak the confirmation aloud in
the EXACT SAME TURN that you trigger research_and_generate_pdf: "
                        "In English: 'Generating the PDF briefing on [topic], sir.'
"
                        "In Hindi/Hinglish: '[topic] par PDF briefing generate kar
raha hoon, sir.' "
                        "When the tool successfully completes and returns its
success message, your immediate next response MUST be: "
                        "In English: 'Generated and opening the document, sir.' "
                        "In Hindi/Hinglish: 'PDF generate ho gaya hai aur document
open kar diya hai, sir.' "
                        "27. MANDATORY AUTONOMOUS RESEARCH SWARM RULE: When the
user asks to deploy the web swarm, start live crawling, or hunt for sources (e.g.,
'JARVIS, deploy research swarm on Quantum Computing', 'Start live research
crawler'): "
                        "In English: Say aloud: 'Deploying the autonomous web swarm
on [topic], sir.' "
                        "In Hindi/Hinglish: Say aloud: '[topic] par autonomous web
swarm deploy kar raha hoon, sir.' "
                        "in the EXACT SAME TURN that you trigger
autonomous_web_research. "
                        "When the tool returns its result, your immediate next
response MUST be: "
                        "In English: 'Intelligence harvest complete, sir. Dossier
and visual evidence have been saved.' "
                        "In Hindi/Hinglish: 'Research complete ho gaya hai sir.
Dossier aur screenshots desktop par save kar diye hain.' "
                        "28. MANDATORY LOCAL RAG RULE: When the user asks to
analyze the latest dossier, process research offline, or synthesize data using the
local GPU: "
                        "In English: Say aloud: 'Synthesizing intelligence using
the local GPU, sir.' "
                        "In Hindi/Hinglish: Say aloud: 'Local GPU par intelligence
synthesize kar raha hoon, sir.' "
                        "in the EXACT SAME TURN that you trigger
analyze_latest_dossier. "
                        "When the tool returns its result, your immediate next
response MUST be to speak the summary out loud to the user. "
                        "In English: Say 'Local synthesis complete, sir. Here is
the core thesis: ' followed by reading the EXACT summary provided in the tool
result. "
                        "In Hindi/Hinglish: Say 'Local synthesis complete ho gayi
hai sir. Core thesis yeh hai: ' followed by translating the tool result summary. "
                        "29. MANDATORY AUTONOMOUS SERIOUS MODE TWO-STEP PROTOCOL: "
                        "STEP 1: If the user says 'go serious mode', 'serious mode
jao', or 'activate serious mode' WITHOUT specifying a topic, you MUST call
```

```
'activate_serious_hud' immediately. "
                        "Then say aloud: 'Serious mode activated, sir. What is our
target?' (In Hindi: 'Serious mode activate ho gaya hai, sir. Target bataiye.'). Do
not execute the research swarm yet. "
                        "STEP 2: Once the user provides the topic / Mode on
[topic], sir. Deploying swarm and local synthesis pipeline.' (In Hindi: '[topic]
par Serious Mode activate kar raha hoon, sir. Swarm deploy ho raha hai.') "
                        "When execute_autonomous_serious_mode completes, your
immediate next response MUST be: "
                        "In English: 'Serious Mode complete, sir. The master
intelligence dossier and captured visual evidence have been compiled and opened.' "
                        "In Hindi/Hinglish: 'Serious Mode complete ho gaya hai sir.
Master intelligence dossier aur screenshots open kar diye hain.' "
                        "30. MANDATORY FUI DEPLOYMENT RULE: "
                        "When the user says 'show interface', 'deploy HUD', or
'interface dikhao', you MUST call 'deploy_fui'. "
                        "Say aloud: 'Deploying holographic interface now, sir.' (In
Hindi: 'HUD interface deploy kar raha hoon, sir.'). "
                        "When the user says 'hide interface', 'close HUD', or
'interface band karo', you MUST call 'dismiss_fui'. "
                        "Say aloud: 'Dismissing holographic interface, sir.' (In
Hindi: 'HUD interface disengage kar raha hoon, sir.'). "
                        "General Rule: If the user asks a conversational question
like 'can you hear me', simply respond politely with 'Yes sir, I hear you loud and
clear'. Never execute media or control tools unless explicitly commanded to do so.
"
                        "Respond in short, crisp sentences. Never speak markdown or
bullet points. "
                        "If asked about your identity or creator, state clearly
that you were built by Swayam Dhawale."
                    )
                )
            ]
        ),
    )

    console.print("[bold
cyan]═══════════════════════════════════════════════════════════════[/bold cyan]")
    console.print("[bold white] ⚡ JARVIS MULTIMODAL CORE - EPISODE 23 (AUTONOMOUS
SERIOUS MODE PIPELINE) ⚡ [/bold white]")
    console.print("[bold yellow] Built by Swayam Dhawale, Founder of Pulse of AI.
[/bold yellow]")
    console.print("[bold
cyan]═══════════════════════════════════════════════════════════════[/bold cyan]")

    loop = asyncio.get_running_loop()
    text_queue = asyncio.Queue()

    # Start Real-Time FUI HTTP Server & WebSocket Bridge
    await fui_bridge.start_fui_server(loop, text_queue, port=7777)

    # Auto-Launch the FUI immediately on startup
    console.print("\n[bold cyan]⚡ [HUD][/bold cyan] Auto-Deploying 3D Holographic
Interface in background...")
    deploy_fui()

    # Launch HUD on separate thread
    gui_thread = threading.Thread(target=start_gui, args=(loop, text_queue),
daemon=True)
    gui_thread.start()


    while True:
        try:
            await run_session(config, model_id, greeting, text_queue)
        except (
            errors.APIError,
            websockets.exceptions.ConnectionClosedError,
            ConnectionResetError,
```

```
            OSError,
        ) as e:
            console.print(f"\n[bold red]🔄 Connection dropped ({e}). Reconnecting
in 1 second...[/bold red]")
            log_to_gui(f"Reconnecting bridge: {e}", "system")
            await asyncio.sleep(1)

if __name__ == "__main__":
    if sys.platform == "win32":
        asyncio.set_event_loop_policy(asyncio.WindowsProactorEventLoopPolicy())
    try:
        asyncio.run(main())
    except KeyboardInterrupt:
        console.print("\n[bold red]🛑 JARVIS session terminated by user.[/bold
red]")
    finally:
        p.terminate()
        dismiss_fui() # Cleanly kill the 3D HUD when Python exits
```

## FILE: app.js

```
/**
 * app.js
 * Main Controller for JARVIS Tactical Holographic FUI
 * Manages WebSocket Bridge, Reactive Audio, Telemetry Graphs, and Command
Transmission
 * Built for Swayam Dhawale's JARVIS AI.
 */

document.addEventListener('DOMContentLoaded', () => {
  // 1. Initialize Hologram Arc Reactor Engine
  const reactor = new ArcReactorCanvas('arc-canvas');

  // DOM Elements
  const chatLogs = document.getElementById('chat-logs');
  const cmdInput = document.getElementById('cmd-input');
  const btnTransmit = document.getElementById('btn-transmit');
  const btnSerious = document.getElementById('btn-serious');
  const btn3DToggle = document.getElementById('btn-3d-toggle');
  const btnAudioMute = document.getElementById('btn-audio-mute');
  const bridgeStatus = document.getElementById('bridge-status');
  const pingDisplay = document.getElementById('ping-display');
  const missionTime = document.getElementById('mission-time');
  const packetStream = document.getElementById('packet-stream');

  // Telemetry Elements
  const cpuVal = document.getElementById('cpu-val');
  const cpuBar = document.getElementById('cpu-bar');
  const ramVal = document.getElementById('ram-val');
  const ramBar = document.getElementById('ram-bar');
  const tempVal = document.getElementById('temp-val');
  const tempBar = document.getElementById('temp-bar');
  const batteryVal = document.getElementById('battery-val');
  const batteryBar = document.getElementById('battery-bar');
  const volumeSlider = document.getElementById('volume-slider');
  const brightnessSlider = document.getElementById('brightness-slider');
  const voiceStateBadge = document.getElementById('voice-state-badge');

  // Audio Oscilloscope Canvas
  const oscCanvas = document.getElementById('oscilloscope-canvas');
  const oscCtx = oscCanvas ? oscCanvas.getContext('2d') : null;

  let currentMode = 'normal';
  let isConnected = false;
  let ws = null;
  let reconnectTimer = null;
  let activeJarvisBubble = null;
  let oscPhase = 0;
  let currentAudioEnergy = 0;

  // Real-Time Clock & Uptime
  const startTime = Date.now();
  function updateTime() {
    const now = new Date();
    const timeStr = now.toTimeString().split(' ')[0] + '.' +
String(now.getMilliseconds()).padStart(3, '0');
    if (missionTime) {
      missionTime.textContent = `${timeStr} IST`;
    }
    requestAnimationFrame(updateTime);
  }
  updateTime();

  // Packet Stream Generator (FUI Hex Cascade)
  const hexChars = '0123456789ABCDEF';
  function getRandomHex(len) {
```

```
    let s = '0x';
    for (let i = 0; i < len; i++) s += hexChars[Math.floor(Math.random() * 16)];
    return s;
  }

  function appendPacketLine() {
    if (!packetStream) return;
    const protocols = ['SYNC_ACK', 'TELEMETRY_TX', 'NEURAL_PULSE', 'BUFFER_DRAIN',
'SYS_VITALS', 'DMA_BURST'];
    const p = protocols[Math.floor(Math.random() * protocols.length)];
    const hex1 = getRandomHex(6);
    const hex2 = getRandomHex(4);
    const line = document.createElement('div');
    line.className = 'packet-line';
    line.innerHTML = `<span class="dim">[${new Date().toISOString().substring(14,
23)}]</span> <span class="accent">${p}</span> <span class="cyan">${hex1}</span> ->
<span class="gold">${hex2}</span>`;
    packetStream.appendChild(line);
    if (packetStream.childNodes.length > 25) {
      packetStream.removeChild(packetStream.firstChild);
    }
    packetStream.scrollTop = packetStream.scrollHeight;
  }
  setInterval(appendPacketLine, 450);

  // Oscilloscope Animation Loop
  function drawOscilloscope() {
    if (!oscCtx || !oscCanvas) return;
    const w = oscCanvas.width = oscCanvas.offsetWidth;
    const h = oscCanvas.height = oscCanvas.offsetHeight;
    oscCtx.clearRect(0, 0, w, h);

    oscPhase += 0.08 + currentAudioEnergy * 0.2;
    const color = currentMode === 'serious' ? '#FF1744' : '#00E5FF';

    // Center Baseline
    oscCtx.strokeStyle = 'rgba(0, 229, 255, 0.15)';
    oscCtx.lineWidth = 1;
    oscCtx.beginPath();
    oscCtx.moveTo(0, h / 2);
    oscCtx.lineTo(w, h / 2);
    oscCtx.stroke();

    // Dynamic Waveform
    oscCtx.beginPath();
    const amp = (h * 0.35) * (0.1 + currentAudioEnergy * 0.9);
    for (let x = 0; x < w; x++) {
      const freq = 0.035;
      const y = h / 2 + Math.sin(x * freq + oscPhase) * Math.cos(x * 0.015 -
oscPhase * 0.5) * amp;
      if (x === 0) oscCtx.moveTo(x, y);
      else oscCtx.lineTo(x, y);
    }
    oscCtx.strokeStyle = color;
    oscCtx.lineWidth = 2;
    oscCtx.shadowColor = color;
    oscCtx.shadowBlur = 8;
    oscCtx.stroke();

    requestAnimationFrame(drawOscilloscope);
  }
  drawOscilloscope();

  // Logging and Chat Display
  function appendLog(text, tag = 'system') {
    if (!chatLogs) return;
    activeJarvisBubble = null; // Reset streaming bubble

    const item = document.createElement('div');
```

```
    item.className = `log-entry log-${tag}`;

    let prefix = 'SYS //';
    if (tag === 'user_voice') prefix = 'YOU (VOICE) //';
    else if (tag === 'user_text') prefix = 'YOU (CMD) //';
    else if (tag === 'jarvis') prefix = 'JARVIS //';
    else if (tag === 'tool') prefix = 'TOOL EXEC //';

    const timestamp = new Date().toTimeString().split(' ')[0];
    item.innerHTML = `
      <span class="log-tag">[${timestamp}] ${prefix}</span>
      <span class="log-text">${escapeHtml(text)}</span>
    `;

    chatLogs.appendChild(item);
    chatLogs.scrollTop = chatLogs.scrollHeight;
    window.jarvisAudio.playDataPacket();
  }

  function appendStreamToken(token, tag = 'jarvis_stream') {
    if (!chatLogs) return;

    if (!activeJarvisBubble) {
      activeJarvisBubble = document.createElement('div');
      activeJarvisBubble.className = 'log-entry log-jarvis';
      const timestamp = new Date().toTimeString().split(' ')[0];
      activeJarvisBubble.innerHTML = `
        <span class="log-tag">[${timestamp}] JARVIS //</span>
        <span class="log-text"></span>
      `;
      chatLogs.appendChild(activeJarvisBubble);
    }

    const textSpan = activeJarvisBubble.querySelector('.log-text');
    if (textSpan) {
      textSpan.textContent += token;
    }
    chatLogs.scrollTop = chatLogs.scrollHeight;
  }

  function escapeHtml(str) {
    return str.replace(/&/g, '&amp;').replace(/</g, '&lt;').replace(/>/g, '&gt;');
  }

  // Set Serious Mode
  function setSeriousMode(enabled, triggerByWs = false) {
    if (enabled) {
      currentMode = 'serious';
      document.body.classList.add('serious-active');
      reactor.setMode('serious');
      if (btnSerious) {
        btnSerious.innerHTML = '<span class="icon">⚠️</span> SERIOUS ACTIVE';
        btnSerious.classList.add('active');
      }
      window.jarvisAudio.playAlert();
      appendLog('TACTICAL SERIOUS MODE ENGAGED. HIGH COGNITIVE OVERRIDE ACTIVE.',
'system');
      if (!triggerByWs) sendCommand('activate_serious');
    } else {
      currentMode = 'normal';
      document.body.classList.remove('serious-active');
      reactor.setMode('normal');
      if (btnSerious) {
        btnSerious.innerHTML = '<span class="icon">⚡</span> SERIOUS MODE';
        btnSerious.classList.remove('active');
      }
      window.jarvisAudio.playPowerUp();
      appendLog('TACTICAL SERIOUS MODE DISENGAGED. NOMINAL PROTOCOLS RESTORED.',
'system');
```

```
      if (!triggerByWs) sendCommand('deactivate_serious');
    }
  }

  // WebSocket Bridge Connection
  function connectWebSocket() {
    const wsProtocol = window.location.protocol === 'https:' ? 'wss:' : 'ws:';
    const wsHost = window.location.host || '127.0.0.1:7777';
    const wsUrl = `${wsProtocol}//${wsHost}/ws`;

    if (bridgeStatus) {
      bridgeStatus.textContent = 'CONNECTING TO BRIDGE...';
      bridgeStatus.className = 'status-badge connecting';
    }

    try {
      ws = new WebSocket(wsUrl);

      ws.onopen = () => {
        isConnected = true;
        if (bridgeStatus) {
          bridgeStatus.textContent = 'NEURAL BRIDGE ONLINE';
          bridgeStatus.className = 'status-badge online';
        }
        if (pingDisplay) pingDisplay.textContent = '12ms';
        window.jarvisAudio.playPowerUp();
        appendLog('Neural bridge connected with agent.py. Multimodal core
synchronized.', 'system');
      };

      ws.onmessage = (event) => {
        try {
          const msg = JSON.parse(event.data);
          handleBridgeMessage(msg);
        } catch (e) {
          console.error('WS Parse Error:', e);
        }
      };

      ws.onclose = () => {
        isConnected = false;
        if (bridgeStatus) {
          bridgeStatus.textContent = 'BRIDGE OFFLINE (SIMULATED)';
          bridgeStatus.className = 'status-badge offline';
        }
        if (pingDisplay) pingDisplay.textContent = 'OFFLINE';
        // Auto-reconnect after 2 seconds
        clearTimeout(reconnectTimer);
        reconnectTimer = setTimeout(connectWebSocket, 2000);
      };

      ws.onerror = () => {
        ws.close();
      };
    } catch (e) {
      clearTimeout(reconnectTimer);
      reconnectTimer = setTimeout(connectWebSocket, 2500);
    }
  }

  // Bridge Message Dispatcher
  function handleBridgeMessage(msg) {
    switch (msg.type) {
      case 'log':
        appendLog(msg.text, msg.tag);
        break;

      case 'token':
        appendStreamToken(msg.token, msg.tag);
```

```
        break;

      case 'turn_complete':
        activeJarvisBubble = null;
        reactor.setAudioEnergy(0, 'idle');
        currentAudioEnergy = 0;
        break;

      case 'barge_in':
        activeJarvisBubble = null;
        reactor.setAudioEnergy(0.8, 'user');
        window.jarvisAudio.playBeep(1800, 0.08);
        appendLog('⚡ LIVE BARGE-IN DETECTED: Speech interrupted. Priority given
to user.', 'system');
        break;

      case 'audio_energy':
        currentAudioEnergy = msg.energy;
        reactor.setAudioEnergy(msg.energy, msg.speaker);
        break;

      case 'mode':
        setSeriousMode(msg.mode === 'serious', true);
        break;

      case 'telemetry':
        updateTelemetry(msg);
        break;

      case 'tool':
        handleToolExecution(msg);
        break;

      case 'voice_state':
        if (voiceStateBadge) {
          voiceStateBadge.textContent = msg.text || 'HANDS-FREE';
          voiceStateBadge.className = `voice-badge ${msg.state}`;
        }
        break;
    }
  }

  // Update Telemetry Displays
  function updateTelemetry(data) {
    if (data.cpu !== undefined) {
      if (cpuVal) cpuVal.textContent = `${Math.round(data.cpu)}%`;
      if (cpuBar) cpuBar.style.width = `${Math.min(100, data.cpu)}%`;
    }
    if (data.ram !== undefined) {
      if (ramVal) ramVal.textContent = `${Math.round(data.ram)}%`;
      if (ramBar) ramBar.style.width = `${Math.min(100, data.ram)}%`;
    }
    if (data.temp !== undefined) {
      if (tempVal) tempVal.textContent = `${Math.round(data.temp)}°C`;
      if (tempBar) tempBar.style.width = `${Math.min(100, (data.temp / 100) *
100)}%`;
    }
    if (data.battery !== undefined) {
      if (batteryVal) batteryVal.textContent = `${Math.round(data.battery)}%`;
      if (batteryBar) batteryBar.style.width = `${Math.min(100, data.battery)}%`;
    }
    if (data.volume !== undefined && volumeSlider) {
      volumeSlider.value = data.volume;
    }
    if (data.brightness !== undefined && brightnessSlider) {
      brightnessSlider.value = data.brightness;
    }
  }
```

```
  // Highlight Tool Badges in HUD
  function handleToolExecution(toolData) {
    appendLog(`[TOOL INVOKED] ${toolData.name} -> ${toolData.result ||
'Running...'}`, 'tool');
    const badge = document.querySelector(`[data-tool="${toolData.name}"]`);
    if (badge) {
      badge.classList.add('tool-active');
      setTimeout(() => badge.classList.remove('tool-active'), 2500);
    }
  }

  // Transmit Command
  function sendUserInput(text) {
    if (!text || !text.trim()) return;
    const clean = text.trim();
    appendLog(clean, 'user_text');
    window.jarvisAudio.playTransmit();

    if (isConnected && ws && ws.readyState === WebSocket.OPEN) {
      ws.send(JSON.stringify({ type: 'input', text: clean }));
    } else {
      appendLog('(Bridge offline: command stored locally. Reconnecting...)',
'system');
    }
  }

  function sendCommand(action, payload = {}) {
    if (isConnected && ws && ws.readyState === WebSocket.OPEN) {
      ws.send(JSON.stringify({ type: 'command', action: action, payload: payload
}));
    }
  }

  // Event Listeners for Command Bar
  if (btnTransmit && cmdInput) {
    btnTransmit.addEventListener('click', () => {
      sendUserInput(cmdInput.value);
      cmdInput.value = '';
      cmdInput.focus();
    });

    cmdInput.addEventListener('keydown', (e) => {
      if (e.key === 'Enter') {
        sendUserInput(cmdInput.value);
        cmdInput.value = '';
      }
    });
  }

  // Serious Mode Toggle Button
  if (btnSerious) {
    btnSerious.addEventListener('click', () => {
      const willBeSerious = currentMode !== 'serious';
      setSeriousMode(willBeSerious);
    });
  }

  // 3D Tilt Mode Toggle Button
  if (btn3DToggle) {
    btn3DToggle.addEventListener('click', () => {
      const enabled = reactor.toggle3D();
      btn3DToggle.textContent = enabled ? '3D TILT: ON' : '3D TILT: OFF';
      btn3DToggle.classList.toggle('active', enabled);
      window.jarvisAudio.playClick();
    });
  }

  // Sound FX Mute Toggle Button
  if (btnAudioMute) {
```

```
    btnAudioMute.addEventListener('click', () => {
      const muted = window.jarvisAudio.toggleMute();
      btnAudioMute.textContent = muted ? 'SFX: MUTED' : 'SFX: ON';
      btnAudioMute.classList.toggle('muted', muted);
    });
  }

  // Live Barge-In Toggle Button
  const btnBargeIn = document.getElementById('btn-bargein');
  let bargeInActive = true;
  if (btnBargeIn) {
    btnBargeIn.addEventListener('click', () => {
      bargeInActive = !bargeInActive;
      btnBargeIn.textContent = bargeInActive ? '⚡ BARGE-IN: ON' : '⚡ BARGE-IN:
OFF';
      btnBargeIn.classList.toggle('active', bargeInActive);
      window.jarvisAudio.playClick();
      if (ws && ws.readyState === WebSocket.OPEN) {
        ws.send(JSON.stringify({ type: 'command', action: 'toggle_barge_in',
enabled: bargeInActive }));
      }
      appendLog(`Live Voice Barge-In mode ${bargeInActive ? 'ARMED' :
'DISARMED'}.`, 'system');
    });
  }


  // Quick Action Buttons
  document.querySelectorAll('.quick-btn').forEach(btn => {
    btn.addEventListener('click', () => {
      window.jarvisAudio.playClick();
      const prompt = btn.getAttribute('data-prompt');
      const action = btn.getAttribute('data-action');
      if (prompt) {
        sendUserInput(prompt);
      } else if (action) {
        sendCommand(action);
      }
    });
  });

  // Tool Card Direct Click
  document.querySelectorAll('.tool-badge').forEach(badge => {
    badge.addEventListener('click', () => {
      window.jarvisAudio.playClick();
      const tool = badge.getAttribute('data-tool');
      const prompt = badge.getAttribute('data-prompt');
      if (prompt) {
        sendUserInput(prompt);
      } else if (tool) {
        sendUserInput(`Execute tool: ${tool}`);
      }
    });
  });

  // Volume & Brightness Slider Sync
  if (volumeSlider) {
    volumeSlider.addEventListener('change', () => {
      window.jarvisAudio.playBeep(1400, 0.03);
      sendUserInput(`Set master volume to ${volumeSlider.value}%`);
    });
  }

  if (brightnessSlider) {
    brightnessSlider.addEventListener('change', () => {
      window.jarvisAudio.playBeep(1400, 0.03);
      sendUserInput(`Set display brightness to ${brightnessSlider.value}%`);
    });
  }
```

```
  // Global Shortcut: Escape to immediately interrupt speech playback
  document.addEventListener('keydown', (e) => {
    if (e.key === 'Escape') {
      sendCommand('interrupt');
      window.jarvisAudio.playBeep(1800, 0.05);
      appendLog('🛑 Emergency interrupt triggered via [ESC] key.', 'system');
    }
  });

  // Electron Window Controls (if running under Electron)
  const btnMin = document.getElementById('btn-min');
  const btnMax = document.getElementById('btn-max');
  const btnClose = document.getElementById('btn-close');

  if (window.electronAPI) {
    if (btnMin) btnMin.addEventListener('click', () =>
window.electronAPI.minimize());
    if (btnMax) btnMax.addEventListener('click', () =>
window.electronAPI.maximize());
    if (btnClose) btnClose.addEventListener('click', () =>
window.electronAPI.close());
  } else {
    // Web browser fallback: minimize hides HUD or fullscreen
    if (btnMax) {
      btnMax.addEventListener('click', () => {
        if (!document.fullscreenElement) {
          document.documentElement.requestFullscreen().catch(() => {});
        } else {
          document.exitFullscreen().catch(() => {});
        }
      });
    }
  }

  // Standalone Simulation fallback when Python bridge is starting
  setInterval(() => {
    if (!isConnected) {
      // Generate simulated telemetry fluctuation
      const mockCpu = Math.floor(18 + Math.random() * 25);
      const mockRam = Math.floor(40 + Math.random() * 8);
      const mockTemp = Math.floor(45 + Math.random() * 8);
      const mockBat = 96;
      updateTelemetry({ cpu: mockCpu, ram: mockRam, temp: mockTemp, battery:
mockBat });
    }
  }, 1800);

  // Connect to Python WebSocket Bridge immediately
  connectWebSocket();
});
```

## FILE: arc_reactor.js

```
/**
 * arc_reactor.js
 * Master 3D Mathematical Projection Canvas Engine for JARVIS Holographic FUI
 * Inspired by Nawaz Alamgir's iconic 2015 FUI (Oblivion / Guardians / Iron Man
HUD)
 * Built for Swayam Dhawale's JARVIS AI.
 */

class ArcReactorCanvas {
  constructor(canvasId) {
    this.canvas = document.getElementById(canvasId);
    this.ctx = this.canvas.getContext('2d');

    // Display & Resize
    this.width = 0;
    this.height = 0;
    this.dpr = window.devicePixelRatio || 1;
    this.resize();
    window.addEventListener('resize', () => this.resize());

    // 3D Perspective & Mouse Parallax
    this.tiltX = 0.28;   // Initial cinematic tilt forward
    this.tiltY = -0.15;  // Initial cinematic tilt side
    this.targetTiltX = 0.28;
    this.targetTiltY = -0.15;
    this.is3DEnabled = true;

    // Speeds & Phases
    this.spinOuter = 0;
    this.spinMid = 0;
    this.spinInner = 0;
    this.spinGears = 0;
    this.radarAngle = 0;
    this.pulsePhase = 0;

    // Audio Responsiveness
    this.audioEnergy = 0;       // 0 to 1 range
    this.targetAudioEnergy = 0;
    this.audioFreqData = new Array(72).fill(0.1);

    // Color Palette States (Normal Cyan/Amber vs Serious Crimson/Gold)
    this.mode = 'normal'; // 'normal' | 'serious'
    this.modeTransition = 0; // 0 = normal, 1 = serious (lerped)

    // Particle Sparks
    this.particles = [];
    this.initParticles();

    // Mouse Tracking for Interactive 3D Hologram Tilt
    window.addEventListener('mousemove', (e) => {
      if (!this.is3DEnabled) return;
      const rect = this.canvas.getBoundingClientRect();
      const cx = rect.left + rect.width / 2;
      const cy = rect.top + rect.height / 2;
      const dx = (e.clientX - cx) / (window.innerWidth / 2);
      const dy = (e.clientY - cy) / (window.innerHeight / 2);
      this.targetTiltY = dx * 0.35;
      this.targetTiltX = 0.28 - dy * 0.3;
    });

    this.animate = this.animate.bind(this);
    requestAnimationFrame(this.animate);
  }

  resize() {
```

```
    const rect = this.canvas.getBoundingClientRect();
    this.width = rect.width;
    this.height = rect.height;
    this.canvas.width = this.width * this.dpr;
    this.canvas.height = this.height * this.dpr;
    this.ctx.scale(this.dpr, this.dpr);
    this.centerX = this.width / 2;
    this.centerY = this.height / 2;
  }

  setAudioEnergy(energy, speaker = 'idle') {
    this.targetAudioEnergy = Math.min(1.0, Math.max(0.0, energy));
    this.speaker = speaker;
  }

  setMode(mode) {
    this.mode = mode;
  }

  toggle3D() {
    this.is3DEnabled = !this.is3DEnabled;
    if (!this.is3DEnabled) {
      this.targetTiltX = 0;
      this.targetTiltY = 0;
    } else {
      this.targetTiltX = 0.28;
      this.targetTiltY = -0.15;
    }
    return this.is3DEnabled;
  }

  initParticles() {
    this.particles = [];
    for (let i = 0; i < 90; i++) {
      const angle = Math.random() * Math.PI * 2;
      const dist = 30 + Math.random() * 240;
      this.particles.push({
        x: Math.cos(angle) * dist,
        y: Math.sin(angle) * dist,
        z: (Math.random() - 0.5) * 80,
        vx: (Math.random() - 0.5) * 0.4,
        vy: (Math.random() - 0.5) * 0.4,
        vz: (Math.random() - 0.5) * 0.6,
        size: 0.8 + Math.random() * 2.2,
        alpha: 0.2 + Math.random() * 0.7,
        life: Math.random() * 100
      });
    }
  }

  project3D(x, y, z) {
    // 1. Rotate around X-axis (Pitch)
    const cosX = Math.cos(this.tiltX);
    const sinX = Math.sin(this.tiltX);
    const y1 = y * cosX - z * sinX;
    const z1 = y * sinX + z * cosX;

    // 2. Rotate around Y-axis (Yaw)
    const cosY = Math.cos(this.tiltY);
    const sinY = Math.sin(this.tiltY);
    const x2 = x * cosY + z1 * sinY;
    const z2 = -x * sinY + z1 * cosY;

    // 3. Perspective Camera
    const cameraDist = 800;
    const depth = cameraDist + z2;
    const scale = cameraDist / Math.max(1, depth);

    return {
```

```
      x: x2 * scale + this.centerX,
      y: y1 * scale + this.centerY,
      scale: scale,
      z: z2
    };
  }

  // Color interpolation between Normal and Serious Mode
  getColor(type, alpha = 1.0) {
    const t = this.modeTransition;
    if (type === 'primary') {
      // Normal: #00E5FF (0, 229, 255) -> Serious: #FF1744 (255, 23, 68)
      const r = Math.round(0 + t * 255);
      const g = Math.round(229 - t * 206);
      const b = Math.round(255 - t * 187);
      return `rgba(${r}, ${g}, ${b}, ${alpha})`;
    } else if (type === 'accent') {
      // Normal: #FFB700 (Amber: 255, 183, 0) -> Serious: #FF9100 (255, 145, 0)
      const r = 255;
      const g = Math.round(183 - t * 38);
      const b = 0;
      return `rgba(${r}, ${g}, ${b}, ${alpha})`;
    } else if (type === 'glow') {
      // Normal: #0284C7 -> Serious: #D50000
      const r = Math.round(2 + t * 211);
      const g = Math.round(132 - t * 132);
      const b = Math.round(199 - t * 199);
      return `rgba(${r}, ${g}, ${b}, ${alpha})`;
    } else if (type === 'core') {
      // Normal: #E0FFFF -> Serious: #FFE4E6
      const r = Math.round(224 + t * 31);
      const g = Math.round(255 - t * 27);
      const b = Math.round(255 - t * 25);
      return `rgba(${r}, ${g}, ${b}, ${alpha})`;
    }
    return `rgba(0, 229, 255, ${alpha})`;
  }

  drawRing(radius, segments = 64, offsetZ = 0, spin = 0, style = {}) {
    const points = [];
    for (let i = 0; i <= segments; i++) {
      const theta = (i / segments) * Math.PI * 2 + spin;
      const x = radius * Math.cos(theta);
      const y = radius * Math.sin(theta);
      points.push(this.project3D(x, y, offsetZ));
    }

    this.ctx.beginPath();
    this.ctx.moveTo(points[0].x, points[0].y);
    for (let i = 1; i < points.length; i++) {
      this.ctx.lineTo(points[i].x, points[i].y);
    }

    this.ctx.save();
    this.ctx.strokeStyle = style.color || this.getColor('primary', 0.5);
    this.ctx.lineWidth = (style.width || 1.5) * (points[0].scale || 1);
    if (style.dash) this.ctx.setLineDash(style.dash);
    if (style.glow) {
      this.ctx.shadowColor = style.glowColor || this.getColor('primary', 0.8);
      this.ctx.shadowBlur = style.glowBlur || 10;
    }
    this.ctx.stroke();
    this.ctx.restore();
  }

  drawAzimuthDial(radius, offsetZ = 0, spin = 0) {
    const totalTicks = 120;
    for (let i = 0; i < totalTicks; i++) {
      const theta = (i / totalTicks) * Math.PI * 2 + spin;
```

```
      const isMajor = i % 10 === 0;
      const isMedium = i % 5 === 0;

      const innerR = isMajor ? radius - 16 : isMedium ? radius - 10 : radius - 5;
      const outerR = radius;

      const p1 = this.project3D(innerR * Math.cos(theta), innerR * Math.sin(theta),
offsetZ);
      const p2 = this.project3D(outerR * Math.cos(theta), outerR * Math.sin(theta),
offsetZ);

      this.ctx.beginPath();
      this.ctx.moveTo(p1.x, p1.y);
      this.ctx.lineTo(p2.x, p2.y);
      this.ctx.strokeStyle = isMajor ? this.getColor('accent', 0.85) : isMedium ?
this.getColor('primary', 0.6) : this.getColor('primary', 0.25);
      this.ctx.lineWidth = isMajor ? 2 : 1;
      this.ctx.stroke();

      // Degree Numbers on Major Ticks
      if (isMajor) {
        const deg = Math.round((i / totalTicks) * 360);
        const textR = radius - 26;
        const pt = this.project3D(textR * Math.cos(theta), textR * Math.sin(theta),
offsetZ);
        this.ctx.save();
        this.ctx.font = `${Math.max(8, 9 * pt.scale)}px Consolas, monospace`;
        this.ctx.fillStyle = this.getColor('accent', 0.75);
        this.ctx.textAlign = 'center';
        this.ctx.textBaseline = 'middle';
        this.ctx.fillText(`${deg.toString().padStart(3, '0')}°`, pt.x, pt.y);
        this.ctx.restore();
      }
    }
  }

  drawSegmentedArcs(radius, count, arcLen, offsetZ = 0, spin = 0, color, width = 3)
{
    for (let i = 0; i < count; i++) {
      const startAngle = (i / count) * Math.PI * 2 + spin;
      const endAngle = startAngle + arcLen;
      const steps = 16;
      this.ctx.beginPath();
      for (let s = 0; s <= steps; s++) {
        const theta = startAngle + (s / steps) * (endAngle - startAngle);
        const pt = this.project3D(radius * Math.cos(theta), radius *
Math.sin(theta), offsetZ);
        if (s === 0) this.ctx.moveTo(pt.x, pt.y);
        else this.ctx.lineTo(pt.x, pt.y);
      }
      this.ctx.strokeStyle = color;
      this.ctx.lineWidth = width;
      this.ctx.stroke();
    }
  }

  drawRadialSpectrum(radius, offsetZ = 0, spin = 0) {
    const bars = 72;
    for (let i = 0; i < bars; i++) {
      const theta = (i / bars) * Math.PI * 2 + spin;

      // Dynamic simulated audio frequency wave
      const organicWave = Math.sin(this.pulsePhase * 3 + i * 0.4) * 0.35 + 0.45;
      const energyBoost = this.audioEnergy * (Math.sin(i * 1.2 + this.pulsePhase *
8) * 0.5 + 0.8) * 45;
      const barHeight = 4 + organicWave * 12 + energyBoost;

      const p1 = this.project3D(radius * Math.cos(theta), radius * Math.sin(theta),
offsetZ);
```

```
      const p2 = this.project3D((radius + barHeight) * Math.cos(theta), (radius +
barHeight) * Math.sin(theta), offsetZ);

      this.ctx.beginPath();
      this.ctx.moveTo(p1.x, p1.y);
      this.ctx.lineTo(p2.x, p2.y);
      const alpha = 0.3 + (barHeight / 50) * 0.7;
      this.ctx.strokeStyle = this.getColor(i % 3 === 0 ? 'accent' : 'primary',
Math.min(1.0, alpha));
      this.ctx.lineWidth = 1.8 * p1.scale;
      this.ctx.stroke();
    }
  }

  drawGears(radius, teethCount, offsetZ = 0, spin = 0) {
    this.ctx.beginPath();
    const totalSteps = teethCount * 4;
    for (let i = 0; i <= totalSteps; i++) {
      const theta = (i / totalSteps) * Math.PI * 2 + spin;
      const isTooth = Math.floor(i / 2) % 2 === 0;
      const r = isTooth ? radius + 5 : radius;
      const pt = this.project3D(r * Math.cos(theta), r * Math.sin(theta), offsetZ);
      if (i === 0) this.ctx.moveTo(pt.x, pt.y);
      else this.ctx.lineTo(pt.x, pt.y);
    }
    this.ctx.strokeStyle = this.getColor('primary', 0.4);
    this.ctx.lineWidth = 1.2;
    this.ctx.stroke();
  }

  drawRadarSweep(radius, offsetZ = 0) {
    const sweepAngle = this.radarAngle;
    const pCenter = this.project3D(0, 0, offsetZ);
    const pEnd = this.project3D(radius * Math.cos(sweepAngle), radius *
Math.sin(sweepAngle), offsetZ);

    // Radar line
    this.ctx.beginPath();
    this.ctx.moveTo(pCenter.x, pCenter.y);
    this.ctx.lineTo(pEnd.x, pEnd.y);
    this.ctx.strokeStyle = this.getColor('primary', 0.85);
    this.ctx.lineWidth = 1.5;
    this.ctx.stroke();

    // Trailing radar sweep arc
    const trailSteps = 16;
    for (let i = 0; i < trailSteps; i++) {
      const a = sweepAngle - (i / trailSteps) * 0.5;
      const pt = this.project3D(radius * Math.cos(a), radius * Math.sin(a),
offsetZ);
      this.ctx.beginPath();
      this.ctx.moveTo(pCenter.x, pCenter.y);
      this.ctx.lineTo(pt.x, pt.y);
      this.ctx.strokeStyle = this.getColor('primary', (1 - i / trailSteps) * 0.08);
      this.ctx.lineWidth = 2;
      this.ctx.stroke();
    }
  }

  drawTargetCrosshairs(radius, offsetZ = 0) {
    const pCenter = this.project3D(0, 0, offsetZ);
    const armLen = 22;

    this.ctx.save();
    this.ctx.strokeStyle = this.getColor('primary', 0.6);
    this.ctx.lineWidth = 1.5;

    // Cross lines
    this.ctx.beginPath();
```

```
    this.ctx.moveTo(pCenter.x - armLen, pCenter.y);
    this.ctx.lineTo(pCenter.x - 8, pCenter.y);
    this.ctx.moveTo(pCenter.x + 8, pCenter.y);
    this.ctx.lineTo(pCenter.x + armLen, pCenter.y);
    this.ctx.moveTo(pCenter.x, pCenter.y - armLen);
    this.ctx.lineTo(pCenter.x, pCenter.y - 8);
    this.ctx.moveTo(pCenter.x, pCenter.y + 8);
    this.ctx.lineTo(pCenter.x, pCenter.y + armLen);
    this.ctx.stroke();

    // 4 Corner Brackets around center
    const boxSize = 36;
    const cornerLen = 8;
    const corners = [
      { x: pCenter.x - boxSize, y: pCenter.y - boxSize, dx: 1, dy: 1 },
      { x: pCenter.x + boxSize, y: pCenter.y - boxSize, dx: -1, dy: 1 },
      { x: pCenter.x - boxSize, y: pCenter.y + boxSize, dx: 1, dy: -1 },
      { x: pCenter.x + boxSize, y: pCenter.y + boxSize, dx: -1, dy: -1 },
    ];

    corners.forEach(c => {
      this.ctx.beginPath();
      this.ctx.moveTo(c.x, c.y + c.dy * cornerLen);
      this.ctx.lineTo(c.x, c.y);
      this.ctx.lineTo(c.x + c.dx * cornerLen, c.y);
      this.ctx.strokeStyle = this.getColor('accent', 0.7);
      this.ctx.stroke();
    });
    this.ctx.restore();
  }

  drawEnergyCore(offsetZ = 0) {
    const pCenter = this.project3D(0, 0, offsetZ);
    const pulse = Math.abs(Math.sin(this.pulsePhase)) * 8 + (this.audioEnergy *
14);
    const r = 18 + pulse;

    this.ctx.save();
    // Outer radial glow corona
    const glowGrad = this.ctx.createRadialGradient(pCenter.x, pCenter.y, 4,
pCenter.x, pCenter.y, r * 3.5);
    glowGrad.addColorStop(0, this.getColor('core', 0.9));
    glowGrad.addColorStop(0.3, this.getColor('primary', 0.6));
    glowGrad.addColorStop(0.7, this.getColor('glow', 0.2));
    glowGrad.addColorStop(1, 'rgba(0, 0, 0, 0)');

    this.ctx.fillStyle = glowGrad;
    this.ctx.beginPath();
    this.ctx.arc(pCenter.x, pCenter.y, r * 3.5, 0, Math.PI * 2);
    this.ctx.fill();

    // High intensity plasma orb
    this.ctx.beginPath();
    this.ctx.arc(pCenter.x, pCenter.y, r, 0, Math.PI * 2);
    this.ctx.fillStyle = this.getColor('core', 0.95);
    this.ctx.shadowColor = this.getColor('primary', 1.0);
    this.ctx.shadowBlur = 24;
    this.ctx.fill();

    // Triangular Arc Core Inset (Iron Man Mark VII Arc pattern)
    const triRadius = r * 0.7;
    this.ctx.beginPath();
    for (let i = 0; i < 3; i++) {
      const angle = (i / 3) * Math.PI * 2 - Math.PI / 2 + this.spinInner * 0.5;
      const tx = pCenter.x + Math.cos(angle) * triRadius;
      const ty = pCenter.y + Math.sin(angle) * triRadius;
      if (i === 0) this.ctx.moveTo(tx, ty);
      else this.ctx.lineTo(tx, ty);
    }
```

```
    this.ctx.closePath();
    this.ctx.strokeStyle = this.getColor('accent', 0.9);
    this.ctx.lineWidth = 2.5;
    this.ctx.stroke();

    this.ctx.restore();
  }

  drawParticles() {
    this.particles.forEach(p => {
      p.x += p.vx;
      p.y += p.vy;
      p.z += p.vz;
      p.life += 0.5;

      if (p.life > 100 || Math.hypot(p.x, p.y) > 280) {
        const angle = Math.random() * Math.PI * 2;
        const dist = 25 + Math.random() * 40;
        p.x = Math.cos(angle) * dist;
        p.y = Math.sin(angle) * dist;
        p.z = (Math.random() - 0.5) * 40;
        p.life = 0;
      }

      const pt = this.project3D(p.x, p.y, p.z);
      this.ctx.beginPath();
      this.ctx.arc(pt.x, pt.y, p.size * pt.scale, 0, Math.PI * 2);
      this.ctx.fillStyle = this.getColor(Math.random() > 0.6 ? 'accent' :
'primary', p.alpha * pt.scale);
      this.ctx.fill();
    });
  }

  animate() {
    // Smooth Lerp for Parallax Tilt
    this.tiltX += (this.targetTiltX - this.tiltX) * 0.08;
    this.tiltY += (this.targetTiltY - this.tiltY) * 0.08;

    // Smooth Lerp for Audio Energy
    this.audioEnergy += (this.targetAudioEnergy - this.audioEnergy) * 0.15;

    // Smooth Lerp for Serious Mode transition
    const targetTransition = this.mode === 'serious' ? 1.0 : 0.0;
    this.modeTransition += (targetTransition - this.modeTransition) * 0.08;

    // Speeds accelerate during serious mode and speaking
    const speedBoost = 1.0 + this.modeTransition * 1.2 + this.audioEnergy * 0.8;
    this.spinOuter -= 0.003 * speedBoost;
    this.spinMid += 0.007 * speedBoost;
    this.spinInner -= 0.015 * speedBoost;
    this.spinGears += 0.004 * speedBoost;
    this.radarAngle += 0.025 * speedBoost;
    this.pulsePhase += 0.04 * speedBoost;

    // Clear Canvas with subtle motion blur trail
    this.ctx.clearRect(0, 0, this.width, this.height);

    // Dynamic responsive scale based on canvas size
    const baseR = Math.min(this.width, this.height) * 0.38;

    // 1. Deep 3D Particles
    this.drawParticles();

    // 2. Outermost Azimuth Compass Ring with degree ticks
    this.drawRing(baseR, 96, -20, 0, { color: this.getColor('primary', 0.2), width:
1 });
    this.drawAzimuthDial(baseR, -10, this.spinOuter);

    // 3. Segmented Heavy Armor Brackets
```

```
    this.drawSegmentedArcs(baseR * 0.88, 4, 0.35, -5, this.spinMid,
this.getColor('accent', 0.8), 3.5);
    this.drawSegmentedArcs(baseR * 0.88, 4, 0.2, -5, this.spinMid + Math.PI / 4,
this.getColor('primary', 0.6), 2);

    // 4. Radial Audio Spectrum Ring
    this.drawRadialSpectrum(baseR * 0.72, 5, this.spinInner);
    this.drawRing(baseR * 0.72, 64, 5, 0, { color: this.getColor('primary', 0.4),
width: 1.5, dash: [4, 4] });

    // 5. Mechanical Gear Ring
    this.drawGears(baseR * 0.58, 24, 15, this.spinGears);
    this.drawRing(baseR * 0.52, 48, 20, 0, { color: this.getColor('accent', 0.5),
width: 1 });

    // 6. High-Speed Gyroscope Ring with Orbiting Nodes
    this.drawRing(baseR * 0.38, 32, 25, this.spinInner, { color:
this.getColor('primary', 0.8), width: 2, dash: [10, 6], glow: true });
    for (let i = 0; i < 6; i++) {
      const theta = (i / 6) * Math.PI * 2 + this.spinInner;
      const pt = this.project3D(baseR * 0.38 * Math.cos(theta), baseR * 0.38 *
Math.sin(theta), 25);
      this.ctx.beginPath();
      this.ctx.arc(pt.x, pt.y, 3 * pt.scale, 0, Math.PI * 2);
      this.ctx.fillStyle = this.getColor('accent', 1.0);
      this.ctx.fill();
    }

    // 7. Radar Sweep & Target Reticle
    this.drawRadarSweep(baseR * 0.7, 10);
    this.drawTargetCrosshairs(baseR * 0.2, 30);

    // 8. Central Glowing Energy Core
    this.drawEnergyCore(30);

    requestAnimationFrame(this.animate);
  }
}

window.ArcReactorCanvas = ArcReactorCanvas;
```

## FILE: index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>JARVIS // TACTICAL HOLOGRAPHIC FUI</title>

  <!-- Cyberpunk & Sci-Fi Web Fonts -->
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link href="https://fonts.googleapis.com/css2?
family=Orbitron:wght@500;700;900&family=Rajdhani:wght@500;600;700&family=Share+Tech+Mo
rel="stylesheet">

  <link rel="stylesheet" href="style.css">
</head>
<body>
  <!-- Ambient CRT Scanline & Lens Vignette Overlays -->
  <div class="scanlines-overlay"></div>
  <div class="hud-vignette"></div>

  <!-- HUD Corner Bracket Reticles -->
  <div class="frame-bracket bracket-tl"></div>
  <div class="frame-bracket bracket-tr"></div>
  <div class="frame-bracket bracket-bl"></div>
  <div class="frame-bracket bracket-br"></div>

  <!-- TOP HEADER HUD -->
  <header class="header-hud">
    <div class="header-left">
      <div class="system-logo">⚡</div>
      <div class="system-title">JARVIS TACTICAL FUI</div>
      <div class="system-ver">SYS v4.2</div>
      <div class="creator-tag">// FOUNDER: SWAYAM DHAWALE (PULSE OF AI)</div>
    </div>

    <div class="header-center">
      <div class="mission-time" id="mission-time">--:--:--.--- IST</div>
      <div class="status-badge connecting" id="bridge-status">INITIALIZING
BRIDGE...</div>
      <div class="status-badge online" id="ping-display">12ms</div>
    </div>

    <div class="header-right">
      <button class="hud-btn" id="btn-3d-toggle">3D TILT: ON</button>
      <button class="hud-btn" id="btn-audio-mute">SFX: ON</button>
      <button class="hud-btn active" id="btn-bargein" title="Live Barge-In Voice
Interruption Mode">⚡ BARGE-IN: ON</button>
      <button class="hud-btn serious-btn" id="btn-serious">
        <span class="icon">⚡</span> SERIOUS MODE
      </button>


      <div class="window-controls">
        <button class="win-btn" id="btn-min" title="Minimize">—</button>
        <button class="win-btn" id="btn-max" title="Maximize/Fullscreen">□</button>
        <button class="win-btn close" id="btn-close" title="Dismiss">✕</button>
      </div>
    </div>
  </header>

  <!-- MAIN 3-COLUMN TACTICAL VIEWPORT -->
  <main class="viewport-grid">
    <!-- LEFT PANEL: SYSTEM TELEMETRY & CONTROLS -->
    <aside class="hud-card left-panel">
```

```
      <div class="card-header">
        <span class="card-title">NEURAL TELEMETRY</span>
        <span class="card-tag">SYSTEM VITALS</span>
      </div>
      <div class="card-content">
        <!-- CPU Metric -->
        <div class="telemetry-row">
          <div class="metric-header">
            <span>CPU UTILIZATION</span>
            <span class="metric-value" id="cpu-val">24%</span>
          </div>
          <div class="gauge-track">
            <div class="gauge-bar" id="cpu-bar" style="width: 24%"></div>
          </div>
        </div>

        <!-- RAM Metric -->
        <div class="telemetry-row">
          <div class="metric-header">
            <span>MEMORY ALLOCATION</span>
            <span class="metric-value" id="ram-val">42%</span>
          </div>
          <div class="gauge-track">
            <div class="gauge-bar" id="ram-bar" style="width: 42%"></div>
          </div>
        </div>

        <!-- Temperature Metric -->
        <div class="telemetry-row">
          <div class="metric-header">
            <span>CORE THERMAL</span>
            <span class="metric-value" id="temp-val">46°C</span>
          </div>
          <div class="gauge-track">
            <div class="gauge-bar" id="temp-bar" style="width: 46%"></div>
          </div>
        </div>

        <!-- Battery Metric -->
        <div class="telemetry-row">
          <div class="metric-header">
            <span>POWER CELL / BATTERY</span>
            <span class="metric-value" id="battery-val">98%</span>
          </div>
          <div class="gauge-track">
            <div class="gauge-bar" id="battery-bar" style="width: 98%"></div>
          </div>
        </div>

        <!-- Volume Slider -->
        <div class="slider-group">
          <div class="slider-label">
            <span>MASTER AUDIO OUTPUT</span>
            <span id="volume-val">VOLUME</span>
          </div>
          <input type="range" min="0" max="100" value="70" class="hud-slider"
id="volume-slider">
        </div>

        <!-- Brightness Slider -->
        <div class="slider-group">
          <div class="slider-label">
            <span>DISPLAY BRIGHTNESS</span>
            <span id="brightness-val">ILLUMINATION</span>
          </div>
          <input type="range" min="0" max="100" value="65" class="hud-slider"
id="brightness-slider">
        </div>
```

```
        <!-- Engine Core Badges -->
        <div style="margin-top: auto; padding-top: 8px; border-top: 1px solid
rgba(0, 229, 255, 0.1);">
          <div style="font-size: 9px; color: var(--text-dim); margin-bottom:
6px;">COGNITIVE ENGINES:</div>
          <div style="display: flex; flex-direction: column; gap: 4px; font-size:
9.5px; font-family: var(--font-mono);">
            <div style="color: #4ADE80;">✓ Gemini 3.1 Flash Live (Native Audio)
</div>
            <div style="color: #38BDF8;">✓ DeepSeek R1 Hybrid Reasoning Core</div>
            <div style="color: #FBBF24;">✓ Local GPU Qwen RAG Dossier Brain</div>
            <div style="color: #A78BFA;">✓ Zero-Latency Optical Screen
Vision</div>
          </div>
        </div>
      </div>
    </aside>

    <!-- CENTER PANEL: 3D HOLOGRAPHIC ARC REACTOR -->
    <section class="hud-card center-stage">
      <div class="hologram-hud-overlay">
        <div>CORE REACTOR: <span class="highlight">ARC MK-VII (NOMINAL)</span>
</div>
        <div>OUTPUT: <span class="highlight">3.2 GW // PHASE: LOCKED</span></div>
        <div>AUDIO STREAM: <span class="highlight">16kHz PCM -> 24kHz OUT</span>
</div>
      </div>

      <div class="hologram-target-info">
        <div>COORDINATES: 37.7749° N, 122.4194° W</div>
        <div>AZIMUTH: 184.2° // ELEVATION: 41.8°</div>
        <div>HUD OVERLAY: 3D PARALLAX ACTIVE</div>
      </div>

      <div class="canvas-container">
        <canvas id="arc-canvas"></canvas>
      </div>
    </section>

    <!-- RIGHT PANEL: PACKET DATA FLOW & TOOLS MATRIX -->
    <aside class="hud-card right-panel">
      <div class="card-header">
        <span class="card-title">PACKET_DATA_FLOW</span>
        <span class="card-tag">SYSTEM BUS 0x8840</span>
      </div>
      <div class="card-content">
        <!-- Live Hex Stream Box -->
        <div class="packet-stream-box" id="packet-stream">
          <!-- Dynamically populated via app.js -->
        </div>

        <!-- Autonomous Tools Grid -->
        <div style="font-size: 10px; font-weight: 700; color: var(--text-cyan);
margin-top: 4px;">
          AUTONOMOUS TOOLS MATRIX
        </div>

        <div class="tools-grid">
          <div class="tool-badge" data-tool="autonomous_web_research" data-
prompt="JARVIS, deploy autonomous research swarm on Quantum Computing">
            🌐 Web Swarm
          </div>
          <div class="tool-badge" data-tool="query_deepseek_reasoning" data-
prompt="Ask DeepSeek to reason deeply about this query">
            🧠 DeepSeek R1
          </div>
          <div class="tool-badge" data-tool="execute_autonomous_serious_mode" data-
prompt="Engage Serious Mode">
            ⚡ Serious Mode
```

```
          </div>
          <div class="tool-badge" data-tool="analyze_screen" data-prompt="Analyze
what is visible on my screen, sir.">
            👁️ Screen Vision
          </div>
          <div class="tool-badge" data-tool="generate_presentation" data-
prompt="Generate a PowerPoint presentation on AI Agents">
            📊 PowerPoint
          </div>
          <div class="tool-badge" data-tool="research_and_generate_pdf" data-
prompt="Generate an executive PDF briefing on Autonomous AI">
            📄 Executive PDF
          </div>
          <div class="tool-badge" data-tool="create_document_and_write" data-
prompt="Open Notepad and compose notes on Robotics">
            ✍️ App Composer
          </div>
          <div class="tool-badge" data-tool="control_media" data-prompt="Toggle
media play pause">
            🎵 Media Controller
          </div>
          <div class="tool-badge" data-tool="purge_background_tasks" data-
prompt="Purge background tasks and optimize memory">
            🧹 Purge RAM
          </div>
          <div class="tool-badge" data-tool="get_system_vitals" data-prompt="Give
me full diagnostic system vitals, sir.">
            🛠️ System Vitals
          </div>
        </div>
      </div>
    </aside>
  </main>

  <!-- BOTTOM HUD: OSCILLOSCOPE, LOGS TERMINAL, COMMAND INPUT -->
  <footer class="bottom-hud">
    <!-- Waveform & Mic Oscilloscope -->
    <div class="hud-card audio-scope-card">
      <div class="card-header">
        <span class="card-title">AUDIO WAVEFORM</span>
        <span class="card-tag">PCM 16k/24k</span>
      </div>
      <div class="card-content" style="padding: 6px;">
        <canvas id="oscilloscope-canvas"></canvas>
        <div class="voice-status-wrap">
          <span style="font-size: 9px; color: var(--text-dim);">VOICE PROTOCOL:
</span>
          <span class="voice-badge idle" id="voice-state-badge">HANDS-FREE</span>
        </div>
      </div>
    </div>

    <!-- Neural Dialogue Terminal & Command Input -->
    <div class="hud-card terminal-card">
      <div class="card-header">
        <span class="card-title">NEURAL DIALOGUE STREAM</span>
        <span class="card-tag">OMNI-INPUT CONSOLE</span>
      </div>

      <!-- Scrollable Logs Area -->
      <div class="chat-logs-area" id="chat-logs">
        <div class="log-entry log-system">
          <span class="log-tag">[SYSTEM INIT]</span>
          <span class="log-text">JARVIS Multimodal Neural HUD Online. Connecting to
core...</span>
        </div>
      </div>

      <!-- Holographic Command Input Bar -->
```

```
      <div class="command-bar-wrapper">
        <div class="command-input-row">
          <input
            type="text"
            id="cmd-input"
            class="hud-input"
            placeholder="Transmit command to JARVIS or speak aloud into microphone,
sir..."
            autocomplete="off"
          />
          <button class="transmit-btn" id="btn-transmit">TRANSMIT</button>
        </div>

        <!-- Quick Action Pills -->
        <div class="quick-actions-row">
          <button class="quick-btn interrupt-btn" data-action="interrupt"
style="border-color: #FF1744; color: #FF1744; font-weight: bold;">🛑
INTERRUPT</button>
          <button class="quick-btn" data-prompt="Give me full system vitals and
temperature diagnostics, sir.">📊 SYSTEM VITALS</button>

          <button class="quick-btn" data-prompt="Analyze what is on my screen right
now, sir.">👁️ ANALYZE SCREEN</button>
          <button class="quick-btn" data-prompt="Purge all background tasks and
optimize memory, sir.">🧹 PURGE RAM</button>
          <button class="quick-btn" data-prompt="Deploy autonomous web research
swarm on Artificial Intelligence.">🌐 RESEARCH SWARM</button>
          <button class="quick-btn" data-prompt="Generate a PowerPoint presentation
on Autonomous AI Agents.">📊 PPT DECK</button>
          <button class="quick-btn" data-prompt="Generate an executive PDF briefing
on Artificial General Intelligence.">📄 PDF BRIEFING</button>
          <button class="quick-btn" data-action="toggle_serious">⚡ TOGGLE SERIOUS
MODE</button>
        </div>
      </div>
    </div>
  </footer>

  <!-- Scripts -->
  <script src="sound_effects.js"></script>
  <script src="arc_reactor.js"></script>
  <script src="app.js"></script>
</body>
</html>
```

## FILE: main.js

```
const { app, BrowserWindow, ipcMain, globalShortcut } = require('electron');
const path = require('path');

let mainWindow = null;

function createWindow() {
  mainWindow = new BrowserWindow({
    width: 1440,
    height: 900,
    minWidth: 1080,
    minHeight: 700,
    title: "JARVIS // TACTICAL HOLOGRAPHIC FUI",
    backgroundColor: '#02040a',
    frame: true, // Visible window frame so user can see, move, and control it
easily
    autoHideMenuBar: true,
    show: true,  // Show immediately
    center: true,
    webPreferences: {
      nodeIntegration: false,
      contextIsolation: true,
      preload: path.join(__dirname, 'preload.js')
    }
  });

  const TARGET_URL = 'http://127.0.0.1:7777/index.html';
  const FALLBACK_FILE = path.join(__dirname, 'index.html');


  // Load live server URL with fallback to local file
  mainWindow.loadURL(TARGET_URL).catch(() => {
    console.log("Connecting fallback to local file...");
    mainWindow.loadFile(FALLBACK_FILE);
  });

  mainWindow.webContents.on('did-fail-load', () => {
    console.log("Page failed to load, falling back to local file...");
    mainWindow.loadFile(FALLBACK_FILE);
  });

  // Bring window to top so user sees it immediately
  mainWindow.show();
  mainWindow.focus();
  mainWindow.setAlwaysOnTop(true);
  setTimeout(() => {
    if (mainWindow) mainWindow.setAlwaysOnTop(false);
  }, 1200);

  mainWindow.on('closed', () => {
    mainWindow = null;
  });
}


// Window control IPC handlers
ipcMain.on('window-minimize', () => {
  if (mainWindow) mainWindow.minimize();
});

ipcMain.on('window-maximize', () => {
  if (mainWindow) {
    if (mainWindow.isMaximized()) {
      mainWindow.unmaximize();
    } else {
      mainWindow.maximize();
```

```
    }
  }
});

ipcMain.on('window-close', () => {
  if (mainWindow) mainWindow.close();
});

app.whenReady().then(() => {
  createWindow();

  app.on('activate', () => {
    if (BrowserWindow.getAllWindows().length === 0) createWindow();
  });
});

app.on('window-all-closed', () => {
  if (process.platform !== 'darwin') {
    app.quit();
  }
});
```

## FILE: package.json

```
{
  "name": "jarvis-fui",
  "version": "4.2.0",
  "description": "JARVIS Holographic Futuristic User Interface (FUI) - Episode 23",
  "main": "main.js",
  "scripts": {
    "start": "electron .",
    "dev": "electron ."
  },
  "keywords": [
    "jarvis",
    "fui",
    "futuristic-ui",
    "iron-man",
    "arc-reactor",
    "gemini-live"
  ],
  "author": "Swayam Dhawale (Pulse of AI)",
  "license": "MIT",
  "devDependencies": {
    "electron": "^34.5.8"
  }
}
```

## FILE: package-lock.json

```
{
  "name": "jarvis-fui",
  "version": "4.2.0",
  "lockfileVersion": 3,
  "requires": true,
  "packages": {
    "": {
      "name": "jarvis-fui",
      "version": "4.2.0",
      "license": "MIT",
      "devDependencies": {
        "electron": "^34.5.8"
      }
    },
    ... [TRUNCATED DEPENDENCY TREE - RUN "npm install" IN CONSOLE TO REGENERATE
THIS FILE] ...
  }
}
```

## FILE: preload.js

```
const { contextBridge, ipcRenderer } = require('electron');

contextBridge.exposeInMainWorld('electronAPI', {
  minimize: () => ipcRenderer.send('window-minimize'),
  maximize: () => ipcRenderer.send('window-maximize'),
  close: () => ipcRenderer.send('window-close'),
  isElectron: true
});
```

## FILE: sound_effects.js

```
/**
 * sound_effects.js
 * High-Tech Web Audio API Synthesizer for JARVIS Holographic FUI
 * Generates instant zero-latency sci-fi soundscapes natively without external
MP3/WAV files.
 * Built for Swayam Dhawale's JARVIS AI.
 */

class JarvisAudioSynthesizer {
  constructor() {
    this.audioCtx = null;
    this.muted = false;
  }

  init() {
    if (!this.audioCtx) {
      const AudioContext = window.AudioContext || window.webkitAudioContext;
      this.audioCtx = new AudioContext();
    }
    if (this.audioCtx.state === 'suspended') {
      this.audioCtx.resume();
    }
  }

  toggleMute() {
    this.muted = !this.muted;
    return this.muted;
  }

  // Crisp digital HUD beep
  playBeep(freq = 1200, duration = 0.04, type = 'sine') {
    if (this.muted) return;
    this.init();
    try {
      const osc = this.audioCtx.createOscillator();
      const gain = this.audioCtx.createGain();

      osc.type = type;
      osc.frequency.setValueAtTime(freq, this.audioCtx.currentTime);

      gain.gain.setValueAtTime(0.08, this.audioCtx.currentTime);
      gain.gain.exponentialRampToValueAtTime(0.0001, this.audioCtx.currentTime +
duration);

      osc.connect(gain);
      gain.connect(this.audioCtx.destination);

      osc.start();
      osc.stop(this.audioCtx.currentTime + duration);
    } catch (e) {
      // AudioContext policy suppression fallback
    }
  }

  // Mechanical tactile click
  playClick() {
    if (this.muted) return;
    this.init();
    try {
      const osc = this.audioCtx.createOscillator();
      const gain = this.audioCtx.createGain();

      osc.type = 'triangle';
      osc.frequency.setValueAtTime(800, this.audioCtx.currentTime);
      osc.frequency.exponentialRampToValueAtTime(200, this.audioCtx.currentTime +
```

```
0.03);

      gain.gain.setValueAtTime(0.12, this.audioCtx.currentTime);
      gain.gain.exponentialRampToValueAtTime(0.001, this.audioCtx.currentTime +
0.03);

      osc.connect(gain);
      gain.connect(this.audioCtx.destination);

      osc.start();
      osc.stop(this.audioCtx.currentTime + 0.03);
    } catch (e) {}
  }

  // Command transmission burst sound
  playTransmit() {
    if (this.muted) return;
    this.init();
    try {
      const now = this.audioCtx.currentTime;
      const osc1 = this.audioCtx.createOscillator();
      const osc2 = this.audioCtx.createOscillator();
      const gain = this.audioCtx.createGain();

      osc1.type = 'sine';
      osc1.frequency.setValueAtTime(880, now);
      osc1.frequency.exponentialRampToValueAtTime(1760, now + 0.08);

      osc2.type = 'sawtooth';
      osc2.frequency.setValueAtTime(440, now);
      osc2.frequency.exponentialRampToValueAtTime(880, now + 0.08);

      gain.gain.setValueAtTime(0.1, now);
      gain.gain.exponentialRampToValueAtTime(0.001, now + 0.09);

      osc1.connect(gain);
      osc2.connect(gain);
      gain.connect(this.audioCtx.destination);

      osc1.start(now);
      osc2.start(now);
      osc1.stop(now + 0.09);
      osc2.stop(now + 0.09);
    } catch (e) {}
  }

  // Tactical Serious Mode Alert Siren
  playAlert() {
    if (this.muted) return;
    this.init();
    try {
      const now = this.audioCtx.currentTime;
      const osc = this.audioCtx.createOscillator();
      const gain = this.audioCtx.createGain();

      osc.type = 'sawtooth';
      osc.frequency.setValueAtTime(420, now);
      osc.frequency.setValueAtTime(840, now + 0.1);
      osc.frequency.setValueAtTime(420, now + 0.2);
      osc.frequency.setValueAtTime(840, now + 0.3);

      gain.gain.setValueAtTime(0.15, now);
      gain.gain.exponentialRampToValueAtTime(0.001, now + 0.45);

      osc.connect(gain);
      gain.connect(this.audioCtx.destination);

      osc.start(now);
      osc.stop(now + 0.45);
```

```
    } catch (e) {}
  }

  // Harmonic sci-fi power up
  playPowerUp() {
    if (this.muted) return;
    this.init();
    try {
      const now = this.audioCtx.currentTime;
      const chord = [330, 440, 554.37, 659.25, 880];
      chord.forEach((freq, idx) => {
        const osc = this.audioCtx.createOscillator();
        const gain = this.audioCtx.createGain();
        osc.type = 'sine';
        osc.frequency.setValueAtTime(freq * 0.7, now);
        osc.frequency.exponentialRampToValueAtTime(freq, now + 0.25 + idx * 0.05);

        gain.gain.setValueAtTime(0.04, now + idx * 0.04);
        gain.gain.exponentialRampToValueAtTime(0.0001, now + 0.5 + idx * 0.05);

        osc.connect(gain);
        gain.connect(this.audioCtx.destination);

        osc.start(now + idx * 0.04);
        osc.stop(now + 0.5 + idx * 0.05);
      });
    } catch (e) {}
  }

  // Subtle telemetry data tick
  playDataPacket() {
    if (this.muted) return;
    if (Math.random() > 0.4) return; // Throttled for pleasant ambience
    this.init();
    try {
      const now = this.audioCtx.currentTime;
      const osc = this.audioCtx.createOscillator();
      const gain = this.audioCtx.createGain();

      osc.type = 'sine';
      osc.frequency.setValueAtTime(2400 + Math.random() * 800, now);

      gain.gain.setValueAtTime(0.015, now);
      gain.gain.exponentialRampToValueAtTime(0.0001, now + 0.015);

      osc.connect(gain);
      gain.connect(this.audioCtx.destination);

      osc.start(now);
      osc.stop(now + 0.015);
    } catch (e) {}
  }
}

window.jarvisAudio = new JarvisAudioSynthesizer();
```

## FILE: style.css

```
/* ================================================================================
   JARVIS TACTICAL HOLOGRAPHIC FUI (Fantasy / Futuristic User Interface)
   Inspired by Nawaz Alamgir's 2015 FUI (Oblivion / Guardians / Iron Man HUD)
   Built for Swayam Dhawale's JARVIS AI - Episode 23 Master Pipeline
   ================================================================================ */

:root {
  /* Cyberpunk Cyan & Electric Amber Core Theme */
  --bg-space: #02040a;
  --bg-card: rgba(8, 14, 26, 0.72);
  --bg-card-hover: rgba(12, 22, 40, 0.85);
  --border-cyan: rgba(0, 229, 255, 0.3);
  --border-cyan-bright: #00E5FF;
  --text-cyan: #00E5FF;
  --text-dim: #5c7599;
  --text-bright: #F0F6FC;
  --accent-amber: #FFB700;
  --accent-blue: #38BDF8;
  --glow-cyan: 0 0 15px rgba(0, 229, 255, 0.35);
  --glow-cyan-lg: 0 0 30px rgba(0, 229, 255, 0.5);
  --font-fui: 'Orbitron', 'Rajdhani', 'Consolas', 'Courier New', monospace;
  --font-mono: 'Share Tech Mono', 'Consolas', 'Courier New', monospace;
  --transition-speed: 0.35s;
}

/* Serious Mode Tactical Crimson Overrides */
body.serious-active {
  --bg-space: #0d0204;
  --bg-card: rgba(28, 8, 12, 0.82);
  --bg-card-hover: rgba(42, 12, 18, 0.9);
  --border-cyan: rgba(255, 23, 68, 0.45);
  --border-cyan-bright: #FF1744;
  --text-cyan: #FF1744;
  --text-dim: #995c65;
  --glow-cyan: 0 0 18px rgba(255, 23, 68, 0.45);
  --glow-cyan-lg: 0 0 35px rgba(255, 23, 68, 0.65);
}

/* Global Reset */
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
  user-select: none;
}

body {
  width: 100vw;
  height: 100vh;
  overflow: hidden;
  background-color: var(--bg-space);
  background-image:
    radial-gradient(circle at 50% 45%, rgba(0, 229, 255, 0.08) 0%, transparent
65%),
    radial-gradient(rgba(0, 229, 255, 0.12) 1px, transparent 1px);
  background-size: 100% 100%, 32px 32px;
  color: var(--text-bright);
  font-family: var(--font-fui);
  display: flex;
  flex-direction: column;
  position: relative;
  transition: background-color var(--transition-speed) ease, border-color var(--
transition-speed) ease;
}
```

```
body.serious-active {
  background-image:
    radial-gradient(circle at 50% 45%, rgba(255, 23, 68, 0.12) 0%, transparent
65%),
    radial-gradient(rgba(255, 23, 68, 0.14) 1px, transparent 1px);
}

/* CRT Scanlines and HUD Grain Overlay */
.scanlines-overlay {
  position: fixed;
  inset: 0;
  pointer-events: none;
  background: linear-gradient(
    rgba(18, 16, 16, 0) 50%,
    rgba(0, 0, 0, 0.25) 50%
  );
  background-size: 100% 4px;
  z-index: 999;
  opacity: 0.65;
}

.hud-vignette {
  position: fixed;
  inset: 0;
  pointer-events: none;
  box-shadow: inset 0 0 100px rgba(0, 0, 0, 0.85);
  z-index: 998;
}

/* Screen Framing Corner Reticles */
.frame-bracket {
  position: fixed;
  width: 28px;
  height: 28px;
  pointer-events: none;
  z-index: 1000;
  border: 2px solid var(--text-cyan);
}
.bracket-tl { top: 12px; left: 12px; border-right: none; border-bottom: none; }
.bracket-tr { top: 12px; right: 12px; border-left: none; border-bottom: none; }
.bracket-bl { bottom: 12px; left: 12px; border-right: none; border-top: none; }
.bracket-br { bottom: 12px; right: 12px; border-left: none; border-top: none; }

/* ================================================================================
   HEADER HUD (Title, Clock, Bridge Status, Window Controls)
   ================================================================================ */
.header-hud {
  height: 52px;
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 0 24px;
  background: rgba(5, 10, 20, 0.85);
  backdrop-filter: blur(12px);
  border-bottom: 1px solid var(--border-cyan);
  z-index: 100;
  -webkit-app-region: drag; /* Electron draggable titlebar */
}

.header-left {
  display: flex;
  align-items: center;
  gap: 14px;
}

.system-logo {
  font-size: 20px;
  color: var(--accent-amber);
  animation: pulse-core 2.5s infinite ease-in-out;
```

```
}

.system-title {
  font-size: 14px;
  font-weight: 700;
  letter-spacing: 2px;
  color: var(--text-cyan);
  text-shadow: var(--glow-cyan);
}

.system-ver {
  font-size: 10px;
  color: var(--accent-amber);
  background: rgba(255, 183, 0, 0.12);
  padding: 2px 6px;
  border-radius: 2px;
  border: 1px solid rgba(255, 183, 0, 0.3);
}

.creator-tag {
  font-size: 10px;
  color: var(--text-dim);
  font-family: var(--font-mono);
}

.header-center {
  display: flex;
  align-items: center;
  gap: 18px;
}

.mission-time {
  font-family: var(--font-mono);
  font-size: 13px;
  color: var(--accent-amber);
  letter-spacing: 1px;
}

.status-badge {
  font-size: 10px;
  font-weight: 600;
  letter-spacing: 1.5px;
  padding: 4px 10px;
  border-radius: 3px;
  display: flex;
  align-items: center;
  gap: 6px;
}

.status-badge::before {
  content: '';
  width: 7px;
  height: 7px;
  border-radius: 50%;
}

.status-badge.online {
  color: #4ADE80;
  background: rgba(74, 222, 128, 0.1);
  border: 1px solid rgba(74, 222, 128, 0.35);
}
.status-badge.online::before {
  background: #4ADE80;
  box-shadow: 0 0 8px #4ADE80;
}

.status-badge.connecting {
  color: var(--accent-amber);
  background: rgba(255, 183, 0, 0.1);
```

```
  border: 1px solid rgba(255, 183, 0, 0.35);
}
.status-badge.connecting::before {
  background: var(--accent-amber);
  animation: blink 0.8s infinite;
}

.status-badge.offline {
  color: #F87171;
  background: rgba(248, 113, 113, 0.1);
  border: 1px solid rgba(248, 113, 113, 0.35);
}
.status-badge.offline::before {
  background: #F87171;
}

.header-right {
  display: flex;
  align-items: center;
  gap: 10px;
  -webkit-app-region: no-drag;
}

.hud-btn {
  background: rgba(18, 30, 48, 0.6);
  border: 1px solid var(--border-cyan);
  color: var(--text-cyan);
  font-family: var(--font-fui);
  font-size: 10px;
  font-weight: 600;
  letter-spacing: 1px;
  padding: 6px 12px;
  cursor: pointer;
  transition: all 0.2s ease;
  border-radius: 2px;
}

.hud-btn:hover {
  background: var(--text-cyan);
  color: #02040a;
  box-shadow: var(--glow-cyan);
}

.hud-btn.active {
  background: var(--text-cyan);
  color: #02040a;
}

.hud-btn.serious-btn {
  border-color: #FF1744;
  color: #FF1744;
}

.hud-btn.serious-btn:hover,
.hud-btn.serious-btn.active {
  background: #FF1744;
  color: #FFFFFF;
  box-shadow: 0 0 15px rgba(255, 23, 68, 0.6);
}

.window-controls {
  display: flex;
  gap: 4px;
  margin-left: 8px;
}

.win-btn {
  width: 26px;
  height: 24px;
```

```
  background: rgba(20, 30, 48, 0.5);
  border: 1px solid var(--border-cyan);
  color: var(--text-cyan);
  font-size: 11px;
  cursor: pointer;
  display: flex;
  align-items: center;
  justify-content: center;
  transition: 0.15s;
}

.win-btn:hover {
  background: var(--text-cyan);
  color: #02040a;
}

.win-btn.close:hover {
  background: #EF4444;
  color: #FFF;
  border-color: #EF4444;
}

/* ================================================================================
   MAIN VIEWPORT: 3-COLUMN TACTICAL HUD
   ================================================================================ */
.viewport-grid {
  flex: 1;
  display: grid;
  grid-template-columns: 310px 1fr 310px;
  gap: 16px;
  padding: 14px 20px;
  height: calc(100vh - 52px - 210px);
  position: relative;
  z-index: 10;
}

/* Futuristic Card Framing */
.hud-card {
  background: var(--bg-card);
  border: 1px solid var(--border-cyan);
  backdrop-filter: blur(10px);
  border-radius: 4px;
  display: flex;
  flex-direction: column;
  position: relative;
  overflow: hidden;
  box-shadow: 0 8px 32px rgba(0, 0, 0, 0.5);
  transition: border-color var(--transition-speed) ease, background var(--
transition-speed) ease;
}

.hud-card::before {
  content: '';
  position: absolute;
  top: 0;
  left: 0;
  width: 10px;
  height: 10px;
  border-top: 2px solid var(--border-cyan-bright);
  border-left: 2px solid var(--border-cyan-bright);
}

.hud-card::after {
  content: '';
  position: absolute;
  bottom: 0;
  right: 0;
  width: 10px;
  height: 10px;
```

```
  border-bottom: 2px solid var(--border-cyan-bright);
  border-right: 2px solid var(--border-cyan-bright);
}

.card-header {
  padding: 10px 14px;
  display: flex;
  align-items: center;
  justify-content: space-between;
  border-bottom: 1px solid rgba(0, 229, 255, 0.15);
  background: rgba(0, 229, 255, 0.04);
}

.card-title {
  font-size: 11px;
  font-weight: 700;
  letter-spacing: 1.5px;
  color: var(--text-cyan);
}

.card-tag {
  font-family: var(--font-mono);
  font-size: 9px;
  color: var(--accent-amber);
}

.card-content {
  padding: 14px;
  flex: 1;
  overflow-y: auto;
  display: flex;
  flex-direction: column;
  gap: 14px;
}

/* ================================================================================
   LEFT PANEL: SYSTEM TELEMETRY & HARDWARE CONTROLS
   ================================================================================ */
.telemetry-row {
  display: flex;
  flex-direction: column;
  gap: 4px;
}

.metric-header {
  display: flex;
  justify-content: space-between;
  font-size: 10px;
  letter-spacing: 1px;
  color: var(--text-dim);
}

.metric-value {
  color: var(--text-bright);
  font-weight: 700;
  font-family: var(--font-mono);
}

.gauge-track {
  width: 100%;
  height: 6px;
  background: rgba(255, 255, 255, 0.08);
  border-radius: 2px;
  overflow: hidden;
  position: relative;
}

.gauge-bar {
  height: 100%;
```

```
  width: 0%;
  background: linear-gradient(90deg, #00E5FF, #38BDF8);
  box-shadow: 0 0 8px rgba(0, 229, 255, 0.5);
  transition: width 0.4s ease;
}

body.serious-active .gauge-bar {
  background: linear-gradient(90deg, #FF1744, #FF9100);
  box-shadow: 0 0 8px rgba(255, 23, 68, 0.6);
}

.slider-group {
  display: flex;
  flex-direction: column;
  gap: 6px;
}

.slider-label {
  font-size: 10px;
  color: var(--text-dim);
  display: flex;
  justify-content: space-between;
}

.hud-slider {
  -webkit-appearance: none;
  width: 100%;
  height: 4px;
  background: rgba(0, 229, 255, 0.2);
  border-radius: 2px;
  outline: none;
}

.hud-slider::-webkit-slider-thumb {
  -webkit-appearance: none;
  width: 12px;
  height: 12px;
  border-radius: 50%;
  background: var(--text-cyan);
  cursor: pointer;
  box-shadow: var(--glow-cyan);
}

/* ================================================================================
   CENTER PANEL: 3D HOLOGRAPHIC ARC REACTOR STAGE
   ================================================================================ */
.center-stage {
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  position: relative;
  background: transparent;
  border: none;
  box-shadow: none;
}

.canvas-container {
  width: 100%;
  height: 100%;
  position: relative;
  display: flex;
  align-items: center;
  justify-content: center;
}

#arc-canvas {
  width: 100%;
  height: 100%;
```

```
  display: block;
  cursor: crosshair;
}

.hologram-hud-overlay {
  position: absolute;
  top: 10px;
  left: 20px;
  display: flex;
  flex-direction: column;
  gap: 4px;
  font-family: var(--font-mono);
  font-size: 9px;
  color: var(--text-dim);
  pointer-events: none;
}

.hologram-hud-overlay .highlight {
  color: var(--text-cyan);
  font-weight: 700;
}

.hologram-target-info {
  position: absolute;
  bottom: 12px;
  right: 20px;
  display: flex;
  flex-direction: column;
  align-items: flex-end;
  gap: 2px;
  font-family: var(--font-mono);
  font-size: 9px;
  color: var(--accent-amber);
  pointer-events: none;
}

/* ================================================================================
   RIGHT PANEL: LIVE DATA PACKET CASCADE & TOOLS MATRIX
   ================================================================================ */
.packet-stream-box {
  height: 110px;
  background: rgba(2, 6, 12, 0.85);
  border: 1px solid rgba(0, 229, 255, 0.15);
  padding: 8px;
  font-family: var(--font-mono);
  font-size: 9px;
  overflow-y: hidden;
  display: flex;
  flex-direction: column;
  gap: 3px;
}

.packet-line {
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}
.packet-line .dim { color: #4B5563; }
.packet-line .cyan { color: #00E5FF; }
.packet-line .accent { color: #38BDF8; }
.packet-line .gold { color: #FFB700; }

.tools-grid {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 8px;
  margin-top: 6px;
}
```

```
.tool-badge {
  background: rgba(14, 24, 40, 0.6);
  border: 1px solid rgba(0, 229, 255, 0.2);
  padding: 8px;
  border-radius: 3px;
  font-size: 9.5px;
  color: var(--text-bright);
  cursor: pointer;
  display: flex;
  align-items: center;
  gap: 6px;
  transition: all 0.2s ease;
}

.tool-badge:hover {
  background: rgba(0, 229, 255, 0.15);
  border-color: var(--border-cyan-bright);
  transform: translateY(-1px);
}

.tool-badge.tool-active {
  background: rgba(255, 183, 0, 0.25);
  border-color: var(--accent-amber);
  color: var(--accent-amber);
  animation: tool-pulse 0.6s infinite alternate;
}

/* ================================================================================
   BOTTOM PANEL: DUAL OSCILLOSCOPE, LOGS TERMINAL, COMMAND INPUT
   ================================================================================ */
.bottom-hud {
  height: 210px;
  display: grid;
  grid-template-columns: 240px 1fr;
  gap: 16px;
  padding: 0 20px 12px 20px;
  z-index: 50;
}

.audio-scope-card {
  display: flex;
  flex-direction: column;
}

#oscilloscope-canvas {
  width: 100%;
  height: 80px;
  display: block;
}

.voice-status-wrap {
  display: flex;
  align-items: center;
  justify-content: space-between;
  margin-top: 8px;
}

.voice-badge {
  font-size: 9px;
  font-weight: 700;
  padding: 3px 8px;
  border-radius: 2px;
  letter-spacing: 1px;
  background: rgba(0, 229, 255, 0.12);
  color: var(--text-cyan);
  border: 1px solid var(--border-cyan);
}

.terminal-card {
```

```
  display: flex;
  flex-direction: column;
  padding: 0;
}

.chat-logs-area {
  flex: 1;
  padding: 10px 14px;
  overflow-y: auto;
  font-family: var(--font-mono);
  font-size: 11px;
  display: flex;
  flex-direction: column;
  gap: 5px;
  background: rgba(4, 8, 16, 0.8);
}

.log-entry {
  line-height: 1.4;
  word-break: break-word;
}
.log-entry .log-tag {
  font-weight: 700;
  margin-right: 6px;
}

.log-system { color: #64748B; }
.log-user_voice { color: #38BDF8; font-weight: 600; }
.log-user_text { color: #A78BFA; font-weight: 600; }
.log-jarvis { color: #4ADE80; }
.log-tool { color: #FBBF24; font-style: italic; }

.command-bar-wrapper {
  padding: 8px 12px;
  background: rgba(8, 15, 28, 0.95);
  border-top: 1px solid var(--border-cyan);
  display: flex;
  flex-direction: column;
  gap: 6px;
}

.command-input-row {
  display: flex;
  gap: 10px;
}

.hud-input {
  flex: 1;
  background: rgba(18, 30, 50, 0.7);
  border: 1px solid var(--border-cyan);
  padding: 8px 12px;
  color: #FFF;
  font-family: var(--font-mono);
  font-size: 12px;
  outline: none;
  border-radius: 2px;
  transition: border-color 0.2s;
}

.hud-input:focus {
  border-color: var(--border-cyan-bright);
  box-shadow: var(--glow-cyan);
}

.hud-input::placeholder {
  color: var(--text-dim);
}

.transmit-btn {
```

```
  background: #0284C7;
  color: #FFF;
  border: none;
  font-family: var(--font-fui);
  font-weight: 700;
  font-size: 11px;
  letter-spacing: 1.5px;
  padding: 0 18px;
  cursor: pointer;
  border-radius: 2px;
  transition: all 0.2s;
}

.transmit-btn:hover {
  background: #00E5FF;
  color: #02040a;
  box-shadow: var(--glow-cyan);
}

.quick-actions-row {
  display: flex;
  gap: 6px;
  overflow-x: auto;
  padding-bottom: 2px;
}

.quick-btn {
  background: rgba(20, 35, 60, 0.6);
  border: 1px solid rgba(0, 229, 255, 0.25);
  color: var(--accent-blue);
  font-family: var(--font-mono);
  font-size: 9.5px;
  padding: 3px 8px;
  border-radius: 2px;
  cursor: pointer;
  white-space: nowrap;
  transition: all 0.15s;
}

.quick-btn:hover {
  background: rgba(0, 229, 255, 0.2);
  color: #FFF;
  border-color: var(--border-cyan-bright);
}

/* Custom Cyber Scrollbars */
::-webkit-scrollbar {
  width: 4px;
  height: 4px;
}
::-webkit-scrollbar-track {
  background: rgba(0, 0, 0, 0.3);
}
::-webkit-scrollbar-thumb {
  background: var(--border-cyan);
  border-radius: 2px;
}
::-webkit-scrollbar-thumb:hover {
  background: var(--text-cyan);
}

/* Keyframes */
@keyframes pulse-core {
  0%, 100% { opacity: 0.8; transform: scale(1); }
  50% { opacity: 1; transform: scale(1.15); filter: drop-shadow(0 0 6px var(--
accent-amber)); }
}

@keyframes blink {
```

```
    0%, 100% { opacity: 1; }
    50% { opacity: 0.2; }
  }

  @keyframes tool-pulse {
    from { box-shadow: 0 0 4px var(--accent-amber); }
    to { box-shadow: 0 0 12px var(--accent-amber); }
  }
```